

Highlights

- A new 1,861-image, six-class AvocadoLeaf dataset collected under diverse field conditions across three Vietnamese provinces.
- ConvNeXt-V2 reaches 92.14% accuracy / 0.900 macro F1 with only 3.4M parameters and 2.30 ms GPU inference, the best accuracy-efficiency trade-off among 15 benchmarked architectures.
- Prototypical-network few-shot learning more than doubles fine-tuning accuracy under limited labels (57.4% vs. 28.6% at 20 shots).
- Full-test-set LIME deletion-metric evaluation and a CNN+LIME+vision-language-model pipeline give quantitatively validated, farmer-readable explanations.
- A genuine train-on-A/test-on-B cross-domain evaluation on an independently collected avocado leaf dataset (India) reveals a severe generalisation gap (98.57% to 60.00% binary accuracy), which few-shot adaptation only partially recovers. A complementary failure-mode analysis and a real-time, cross-platform on-device deployment test (Windows, macOS, Android, iOS) further probe generalisation and inference behaviour beyond the GPU test-set benchmark.

A Fine-Grained Dataset and Lightweight Deep Learning Model for Avocado Leaf Disease Classification on Edge Devices

Long Nguyen-Hoang^{a,b}, Dung Ho-Tan^{a,b}, Tien Minh-Do^{a,b,c}

^aFaculty of Information Science and Engineering, University of Information Technology, Ho Chi Minh City, 700000, Vietnam

^bVietnam National University Ho Chi Minh City, Ho Chi Minh City, 700000, Vietnam

^cHo Chi Minh City University of Industry and Trade, Ho Chi Minh City, 700000, Vietnam

Abstract

Avocado leaf diseases cause significant economic losses, yet existing deep learning solutions are often too heavy for edge deployment and lack robustness to real-world variations. This paper introduces the AvocadoLeaf dataset, comprising 1,861 images of five disease categories and healthy leaves, captured under diverse field conditions. Among 15 lightweight and efficient architectures benchmarked on this dataset, ConvNeXt-V2 offers the best accuracy-efficiency trade-off, achieving 92.14% accuracy and 0.900 macro F1 on the test set with only 3.4M parameters at 2.30 ms per image on a single GPU. To handle data scarcity, we evaluate few-shot learning. Prototypical Networks reach 57.4% accuracy with only 20 labelled images per class, substantially outperforming conventional fine-tuning (28.6%). Uncertainty estimation shows the model’s raw softmax confidence is only moderately calibrated (ECE = 0.063), improved to 0.019 via post-hoc temperature scaling at no cost to accuracy, and shows strong out-of-distribution detection (AUC 0.994 for far OOD and 0.985 for near OOD), which translates into markedly safer naive-deployment behaviour than lighter candidate architectures. However, a genuine cross-domain test, training on AvocadoLeaf and evaluating zero-shot on an independently collected, same-task avocado leaf dataset from a different country, reveals a severe generalisation gap: binary (healthy/diseased) accuracy falls from 98.57% in-domain to 60.00%, which few-shot adaptation with a handful of labelled target-domain images only partially recovers (up to 68.78% at 20 shots per class). LIME explanations, quantitatively validated with a deletion metric across the full test set (mean confidence drop 0.649), confirm that the model attends to disease-relevant leaf regions. We further combine the CNN’s prediction with LIME’s visual attribution and a small, free, locally-hosted vision-language model to generate grounded, farmer-readable natural-language explanations, while keeping the CNN as the sole decision-maker throughout. Together, these contributions demonstrate that ConvNeXt-V2 is accurate, data-efficient, interpretable, and a strong candidate for edge deployment in precision agriculture, reliably flagging inputs it should not be trusted on. On-device deployment tests on Android, iOS, Windows, and macOS confirm real-time-capable inference across all four platforms, though a dedicated embedded board such as a Raspberry Pi remains untested, and, as the cross-domain result above shows, further work is needed on generalising to new deployment sites without site-specific adaptation. The code and dataset will be publicly released.

Keywords: Avocado leaf disease, lightweight deep learning, ConvNeXt, few-shot learning, Prototypical Networks, uncertainty estimation, out-of-distribution detection, edge artificial intelligence, precision agriculture

1. Introduction

Avocado (*Persea americana*) is a high-value crop with growing importance in the global agricultural economy, yet its production is constantly threatened by leaf diseases and pest damage, including leaf scorch, leaf spot, algal rust, powdery mildew, and sap-feeding mealybugs, which can severely reduce fruit yield and quality. Early and accurate diagnosis is therefore essential for effective crop management, but in practice it

still relies on visual inspection by human experts, a process that is time-consuming, subjective, and impossible to scale to large orchards.

Deep learning, particularly convolutional neural networks (CNNs), has emerged as a promising alternative, often reporting remarkable accuracy on controlled benchmarks such as PlantVillage. Translating that promise to real farms, however, exposes four gaps that controlled benchmarks tend to hide. First, most high-accuracy models are computationally heavy, requiring

GPUs that are impractical for edge devices such as smartphones or a Raspberry Pi in the field. Second, collecting and annotating thousands of images per disease class is expensive and labour-intensive, while deep models conventionally assume exactly that scale of labelled data is available. Third, a model that reports a class label without any notion of its own uncertainty is unsafe to act on in high-stakes decisions, and offers no signal for when a diagnosis should be deferred to a human expert. Fourth, a black-box prediction, however accurate, does little to build the trust of end-users such as farmers and agronomists, who need to see *why* a leaf was flagged as diseased before they act on it.

This paper addresses all four gaps around a single proposed model, which we name **BoVien**¹, through the following contributions.

1. **AvocadoLeaf dataset.** A new dataset of 1,861 images covering five disease conditions and one healthy class, captured first-hand under diverse real-world conditions (variable lighting, multiple angles, natural backgrounds) rather than in a controlled studio setting. The dataset exhibits moderate class imbalance (max/min ratio 3.96) and fine-grained inter-class similarity (Silhouette score 0.0067, intra/inter distance ratio 1.59), reflecting the genuine difficulty of field-based diagnosis rather than an artificially easy benchmark.
2. **Model BoVien, a lightweight architecture for edge deployment.** Selected from a systematic benchmark of 15 architectures spanning CNN, hybrid CNN-Transformer, and pure-transformer families (Section 6), BoVien (a ConvNeXt-V2 backbone with selective layer fine-tuning) reaches the best accuracy-size-latency trade-off, with 92.14% test accuracy and 0.900 macro F1 at only 3.4M parameters and 2.30 ms per image on a single GPU.
3. **Quantitative, farmer-facing explainability.** A full-test-set LIME deletion-metric evaluation confirms that BoVien’s predictions are grounded in disease-relevant leaf regions rather than background artefacts: hiding the $\sim 20\%$ of the image LIME attributes each prediction to drops the model’s confidence by a mean of 0.649 (95% CI [0.618, 0.678], $n = 280$). Building on this, we combine the CNN’s prediction, LIME’s visual attribution, and a small, free, locally-hosted vision-language model to generate grounded, natural-language explanations of individual predictions,

while keeping the CNN as the sole decision-maker (Section 6.3).

4. **Few-shot learning as a data-efficient fallback.** To probe how the approach degrades when labelled data is scarce, we compare standard fine-tuning against Prototypical Networks (ProtoNet) and Relation Networks. ProtoNet more than doubles fine-tuning’s accuracy at every shot level tested, reaching 57.4% accuracy from only 20 labelled images per class (versus 28.6% for fine-tuning), showing that metric-based few-shot learning is a viable fallback when a new class or a new deployment site starts with little labelled data.
5. **Uncertainty estimation and out-of-distribution (OOD) detection.** Using Monte Carlo Dropout, BoVien separates in-distribution inputs from out-of-distribution ones with high reliability, reaching AUC 0.994 on far-OOD (unrelated internet images) and AUC 0.985 on the harder near-OOD case (leaves of other crop species), giving a practical signal for when a prediction should be flagged for human review. Its raw softmax confidence, by contrast, is only moderately calibrated (ECE = 0.0625), which we discuss as a limitation in Section 7 rather than overstate here. This OOD test asks whether BoVien recognises *off-task* inputs (non-leaf images, leaves of other crops) as unfamiliar. It is a distinct question from whether BoVien still recognises *avocado leaves collected at a new site*, which Section 6.6.2 answers separately and less favourably.

Figure 1 summarises how these contributions fit together, from field data collection through architecture selection to the explainability, robustness, and edge-deployment evaluation that follows.

The remainder of this paper is organised as follows. Section 2 reviews related work in plant disease classification, lightweight edge architectures, explainable and vision-language-based diagnosis, few-shot learning, and uncertainty and cross-dataset generalisation. Section 3 describes the AvocadoLeaf dataset construction process, and Section 4 analyses its statistics, photometric properties, and intrinsic difficulty. Section 5 details the 15-architecture benchmark and the resulting BoVien architecture. Section 6 reports the full experimental results, covering architecture comparison, explainability, few-shot learning, uncertainty estimation, cross-dataset evaluation, and real-time inference. Section 7 discusses findings and limitations, and Section 8 concludes the paper and outlines future work.

¹From Vietnamese *bơ* (avocado) + *viên* (a small unit / device).

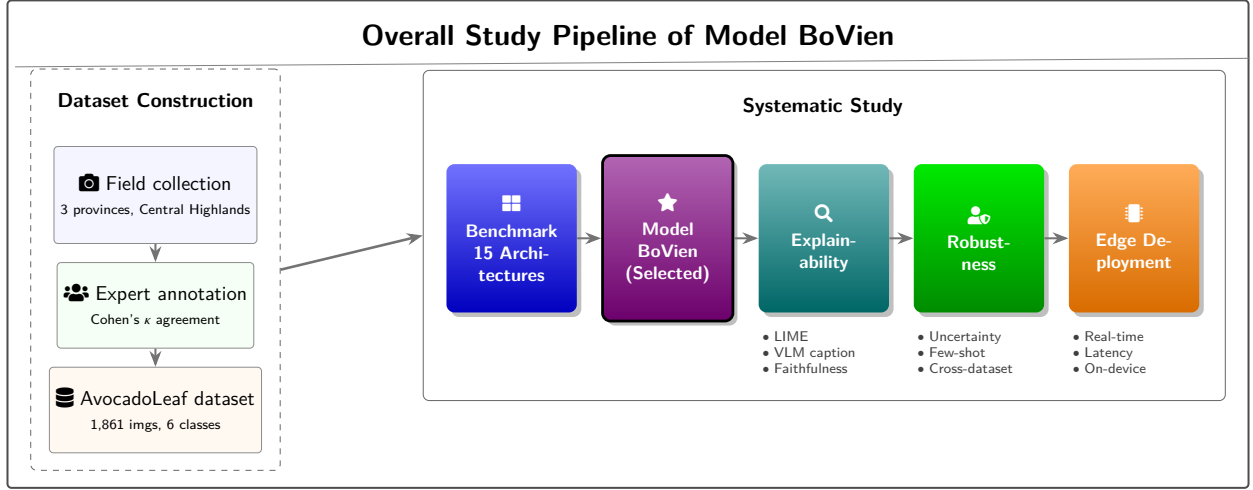


Figure 1: Overview of the overall study pipeline. On the left, AvocadoLeaf dataset construction (field collection across three Central Highlands provinces, expert annotation with inter-annotator agreement, yielding 1,861 quality-filtered images across 6 classes). On the right, the systematic study built on top of it, benchmarking 15 architectures, selecting the proposed Model BoVien, then evaluating it for explainability, robustness (uncertainty/OOD, few-shot, cross-dataset), and edge-deployment readiness.

2. Related Work

Building a plant disease classifier that is accurate, edge-deployable, explainable, and reliable under real field conditions draws on several largely separate literatures. We review each in turn: the datasets and CNN-based classifiers that establish the task (Section 2.1), lightweight architectures for edge deployment (Section 2.2), pixel-level and language-based explanation methods (Sections 2.3–2.4), and the data-efficiency and reliability concerns that determine whether such a classifier can be trusted in practice (Sections 2.5–2.7).

2.1. Plant Disease Classification

Early works on plant disease recognition used traditional machine learning with hand-crafted features. The introduction of deep learning, especially convolutional neural networks (CNNs), has since become the dominant paradigm. The PlantVillage dataset (Hughes and Salathé, 2015) contains more than 50,000 images of healthy and diseased leaves under controlled conditions and underlies a large share of this literature, with models such as VGG, ResNet, and Inception routinely reported near ceiling accuracy on it (Mohanty et al., 2016). Two recent surveys covering work from 2015 through 2023 confirm this pattern at scale, and both single out limited robustness under field conditions as the field’s central open problem, since PlantVillage-level accuracy is understood to reflect the dataset’s controlled imaging rather than a solved recognition problem (Shoib et al., 2023; Mustofa et al.,

2023). PlantDoc (Singh et al., 2020), a smaller (2,598-image), internet-sourced, field-style dataset spanning 13 species on which PlantVillage-pretrained models transfer poorly, gives this concern a concrete empirical footing. Field-collected datasets nonetheless remain scarce more broadly (Barbedo, 2019), and avocado-specific datasets are especially rare. The only one we are aware of, K-Kotagiri (2024), is a 435-image, binary (healthy versus diseased) dataset collected in Tamil Nadu, India, without disease-level labels. AvocadoLeaf is positioned squarely in this gap: 1,861 images spanning five disease categories and a healthy class, captured under diverse natural conditions, including variable lighting, multiple angles, and cluttered backgrounds. Table 1 places it alongside PlantVillage, PlantDoc, and K-Kotagiri along the dimensions that matter here, imaging setting and species/class coverage, rather than raw image count. K-Kotagiri’s independent collection site and labelling scheme also make it the candidate dataset for the cross-domain evaluation in Section 6.

The trade-off is that PlantDoc’s broader 13-species scope comes at the cost of far fewer images per class than a single-crop dataset such as AvocadoLeaf can provide (Section 3). K-Kotagiri, meanwhile, targets the same species but only at the coarse healthy/diseased level, which is why we treat it in Section 6 as a cross-domain *binary* transfer test rather than a like-for-like replacement for AvocadoLeaf’s 6-class taxonomy.

Table 1: Comparison of AvocadoLeaf with three related plant disease benchmarks.

Dataset	Images	Species / classes	Setting
PlantVillage (Hughes and Salathé, 2015)	>50,000	multi-crop	lab-controlled (uniform background, fixed illumination)
PlantDoc (Singh et al., 2020)	2,598	13 species	field, internet-sourced (uncontrolled and uncured)
K-Kotagiri (2024)	435	1 species (avocado), 2 classes (healthy/diseased)	field-collected, single site (Tamil Nadu, India), no disease-level labels
AvocadoLeaf (ours)	1,861	1 species (avocado), 6 classes	field-collected first-hand across 3 provinces (deliberate coverage of light, weather, angle)

2.2. Lightweight Architectures for Edge Deployment

Deploying deep learning models on resource-constrained edge devices (e.g., smartphones, Raspberry Pi) requires careful trade-offs between accuracy, model size, and inference speed. MobileNet (Howard et al., 2017) and EfficientNet (Tan and Le, 2019) are popular families of lightweight CNNs that use depthwise separable convolutions and neural architecture search, while ConvNeXt (Liu et al., 2022), and its V2 revision (Woo et al., 2023) used as the proposed model in this paper (Section 5), instead modernise standard CNNs with design elements borrowed from vision transformers to reach a favourable accuracy-efficiency trade-off. Applications of lightweight models to plant disease classification have often adopted one such family and benchmarked a handful of its variants, e.g. (Atila et al., 2021), rather than comparing across CNN, hybrid CNN-Transformer, and pure transformer families under one shared protocol. Section 6 takes the broader view, benchmarking 15 architectures spanning all three families to identify ConvNeXt-V2 as offering the best accuracy-efficiency trade-off for edge deployment. The shared protocol has one acknowledged exception (the ResNet-18 baseline unfreezes one stage rather than two, Section 5) and one surfaced only after training (an augmentation-policy inconsistency, Section 6.1.2). Section 7 discusses both and why neither is judged likely to change which architecture is selected. Sections 6.6 and 6.7 then go beyond the batched-GPU-latency com-

parisons typical of this literature, probing the chosen architecture’s behaviour under distribution shift and its live, on-device inference latency across four platforms, respectively.

2.3. Explainable AI for Plant Diagnosis

Understanding why a model makes a particular prediction is crucial for building trust with end-users. Local Interpretable Model-agnostic Explanations (LIME) (Ribeiro et al., 2016) is a popular model-agnostic technique for visualising the superpixels of an input image that most influence the model’s decision, while gradient-based methods such as Grad-CAM (Selvaraju et al., 2017) instead produce a coarser class-activation heatmap directly from a CNN’s internal gradients. In agriculture, XAI has been used to highlight disease symptoms on leaves (Sharma et al., 2020), helping farmers verify model outputs, though typically through qualitative visualisation on a handful of hand-picked examples rather than a quantitative measure of how faithful the highlighted region is to the model’s actual decision. Section 6 closes this gap for LIME specifically, applying a deletion metric across the full 280-image test set to test, image by image, whether removing the highlighted region collapses the model’s confidence as a faithful explanation should.

2.4. Vision-Language Models for Agricultural Explanation

Beyond pixel-level attribution, a fast-growing body of work explores large vision-language models (VLMs) for plant diagnosis. One line of work fine-tunes a VLM as the diagnostic decision-maker itself. AgriGPT (Awais et al., 2025) builds an instruction-tuning dataset from agricultural vision-only data to support conversational, fine-grained crop and disease identification, and AgriChain (Mahmood et al., 2026) fine-tunes a VLM on expert-verified, image-grounded diagnostic rationales so the same model predicts the class and produces its supporting explanation. A second, benchmark-oriented line evaluates VLMs on large visual-question-answering (VQA) corpora built specifically for plant science, such as PlantExpertVQA (Sakib et al., 2025) (over 765,000 question-answer pairs across 38 crop species) and a two-stage crop-disease VQA framework that pairs a frozen vision encoder with Grad-CAM and token-level attribution for interpretability (Hossain et al., 2026). Across both lines, the VLM’s own classification or generation ability is what ultimately drives correctness. In contrast, our pipeline keeps a separately validated CNN as the sole decision-maker and uses a small, free, locally-hosted VLM (Qwen2-VL-2B-Instruct, a similar parameter scale to the compact models fine-tuned in Sakib et al. (2025)) only to narrate the CNN’s already-decided prediction together with LIME’s visual attribution (Section 6.3). We additionally report, as a negative result, that letting the same VLM’s judgement double as an automatic error-detection signal for the CNN does not outperform the CNN’s own predictive-entropy baseline (Appendix E).

2.5. Few-shot Learning for Plant Disease Recognition

Collecting large annotated datasets for every possible disease is impractical, which motivates few-shot learning: recognising new classes from only a handful of labelled examples. Popular approaches include metric-based methods such as Prototypical Networks (ProtoNet) (Snell et al., 2017) and Relation Networks (Sung et al., 2018), as well as optimisation-based methods like MAML (Finn et al., 2017), and in plant pathology such methods have been applied to tomato, rice, and cassava diseases (Argüeso et al., 2020; Wang et al., 2021). These applications typically still fine-tune or meta-train an ImageNet-pretrained backbone on the target disease domain, rather than testing a frozen, already-trained classification backbone directly as a metric-learning feature extractor. Section 6.4 takes the latter route, evaluating ProtoNet and Relation Networks with a

frozen ConvNeXt-V2 backbone (already trained for full-shot classification, Section 6) on the fine-grained AvocadoLeaf classes, and compares both against conventional fine-tuning under the same shot budgets.

2.6. Uncertainty Estimation in Deep Learning

High accuracy alone is insufficient for high-stakes applications. A reliable model must also be well calibrated and able to detect out-of-distribution (OOD) inputs. Expected Calibration Error (ECE) (Guo et al., 2017) is a standard metric for assessing calibration. Monte Carlo (MC) Dropout (Gal and Ghahramani, 2016) provides a simple yet effective way to estimate predictive uncertainty by treating dropout at inference time as approximate Bayesian sampling, requiring only one trained model. Deep Ensembles (Lakshminarayanan et al., 2017) instead train several independent models from different random initialisations and combine their predictions, typically yielding better-calibrated uncertainty at the cost of a multiplicative increase in training and storage cost. We adopt MC Dropout over ensembling for this reason, since it reuses the single ConvNeXt-V2 model already trained for classification (Section 6) rather than requiring several independently trained copies. Calibration (Wang et al., 2022) and OOD detection (Gonzalez et al., 2020) have each received some attention in plant disease classification, but rarely together, and rarely for a model already selected for edge deployment on efficiency grounds. Section 6 reports both for the same lightweight avocado-disease model, measuring calibration directly and testing OOD detection against two distinct kinds of shift, far (random internet images) and near (leaves of other crop species), so that the two aspects of reliability can be read against the same accuracy and efficiency numbers established earlier in that section.

2.7. Cross-dataset Generalization in Plant Disease Classification

A model’s accuracy on its own held-out test set does not guarantee it will generalise to images from a different source. Wu et al. (2023) show that networks trained on laboratory-style images suffer marked accuracy loss when applied to field-collected images of the same diseases, and propose unsupervised domain adaptation to narrow this laboratory-to-field gap. Sun (2024) instead trains standard architectures (ResNet, EfficientNet, Inception, MobileNet) on one tomato pest/disease dataset and evaluates them directly on independently collected tomato datasets with no adaptation step, finding substantial accuracy degradation regardless of architecture. Yang et al. (2024) report a similar lab-to-field gap and

propose cross-domain few-shot learning to narrow it, evaluating on their own independently collected Field Crop Disease Dataset. All three studies rely on a second, independently labelled dataset covering the same classes to measure this gap directly as a classification-accuracy drop. While the K-Kotagiri dataset enables an independent binary cross-domain evaluation Section 6.6, we do not yet have a second, full 6-class AvocadoLeaf-like dataset for multi-class disease evaluation. Instead, Section 6.6 reuses the far- and near-OOD image sets already collected for out-of-distribution detection to characterise a related but distinct question, namely how confidently a naive (non-uncertainty-gated) deployment would misclassify domain-shifted, non-avocado inputs. Collecting an independent, same-task dataset to enable a true cross-dataset accuracy evaluation in the sense of (Wu et al., 2023; Sun, 2024; Yang et al., 2024) is left as future work.

3. Dataset Construction

3.1. Data Collection and Filtering

We refer to the resulting dataset as **AvocadoLeaf** throughout this paper. All images were collected by our team in avocado orchards across three provinces of the Central Highlands (Gia Lai, Đắk Lắk, and Đắk Nông). We visited multiple gardens to capture the natural diversity of leaf appearance across (1) light (full sun, cloudy, backlight), (2) weather (dry, cloudy, sunny, after rain), (3) time of day (morning, noon, afternoon), and (4) angle and distance (top-down, oblique, close-up).

To ensure quality, we applied the following filtering steps:

- (1) Removal of duplicate images.
- (2) Exclusion of images with motion blur so severe that the leaf becomes impossible to distinguish.
- (3) Manual screening to discard frames where the avocado leaf is occluded more than 50% or is otherwise impossible to identify reliably.

After filtering, 1,861 images remained and were passed to the annotation phase.

3.2. Annotation Guideline and Metadata Definition

We defined a simple annotation protocol for the six avocado leaf conditions observed in our dataset. The six class labels and their meanings are:

- **binhthuong**: healthy leaf, no visible damage or disease.

- **chaymepla**: leaf scorch, an abiotic physiological disorder with no causal pathogen, tip or margin burn typically caused by salt stress or sunburn.
- **domla**: *Cercospora* leaf spot (*Pseudocercospora purpurea*), small brown to purplish-black lesions caused by fungal infection.
- **gisat**: algal leaf spot, locally known as “red rust” (*Cephaleuros virescens*), orange, velvety pustule-like spots caused by a parasitic alga.
- **moctrang**: powdery mildew (*Oidium* sp., family Erysiphaceae), white powdery coating on the leaf surface.
- **repsap**: long-tailed mealybug damage (*Pseudococcus longispinus*), sap-feeding insect leaving silver or stippled marks.

Figure 2 shows one representative image per class.

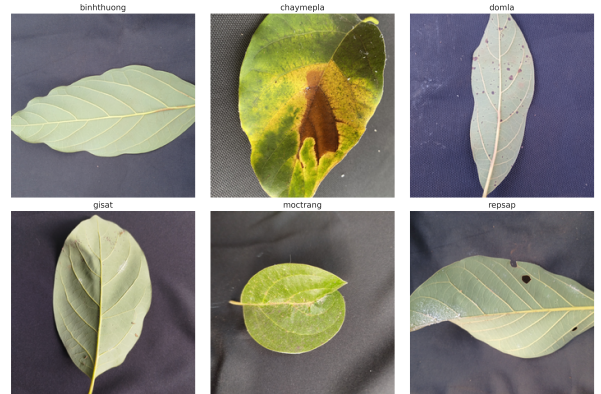


Figure 2: Representative sample image for each of the six AvocadoLeaf classes.

The annotators followed a consistent guideline:

- Each image must contain at least one clearly visible avocado leaf as the main subject.
- The label is assigned based on the dominant visible symptom on that leaf. If multiple symptoms are present, the annotator selects the most severe condition. This collapses any co-occurring secondary symptom into the single-label taxonomy used throughout this work. We return to this source of label ambiguity in Section 7.
- Images where the leaf cannot be reliably identified (e.g., extreme blur, heavy occlusion) are discarded during manual screening.

All annotations were recorded in a simple CSV file with `filename` and `label` columns. No bounding boxes or additional metadata were required, as the task is image-level classification.

3.3. Quality Control

To ensure the reliability of the annotated dataset, we conducted a quality control step using an independent expert review.

Annotation aggregation. After three annotators independently labelled the entire dataset, the final label for each image was determined by majority voting. Pairwise agreement among the three annotators was monitored throughout as the trigger for the re-labelling loop shown in Figure 3. A round was to be repeated whenever pairwise agreement fell below a pre-set acceptable threshold. This threshold was never crossed, so no round was ever repeated, and no per-round inter-annotator Kappa values were retained for reporting here. This is a documented limitation of the protocol. The actual pairwise agreement values are not available for independent verification. In the rare case of a three-way tie, where all three annotators disagree, the image was flagged for a short discussion to reach a unanimous decision. No such case occurred in practice. As a result, Appendix A reports each annotator’s individual agreement with the independent expert (Table A.14) but not a pairwise Kappa among the three annotators themselves. The Cohen’s Kappa reported later in this section should therefore be read as validating the consensus label against an external reference. It is not a measure of how much the three annotators agreed with each other before that consensus was formed.

Expert review procedure. From the 1,861 images with their consensus labels, we randomly sampled 15% (279 images) for this review, drawn independently of the train/validation/test split used for model development. This review sample and the 279-image validation split (Section 4) coincide in size only because both happen to be a 15% draw from the same 1,861-image pool. These images were presented to an avocado leaf disease expert who had not participated in the original annotation. The expert, blind to the annotators’ decision, assigned one of the six predefined classes to each image following the same annotation guideline. As only one expert was involved, the resulting agreement reflects consistency with a single reviewer’s judgment rather than inter-expert reliability.

Agreement metric. We measured the agreement between the annotator consensus labels and the expert labels using Cohen’s Kappa coefficient (κ), which accounts for chance agreement. It is defined as:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (1)$$

where p_o is the observed proportion of agreement and p_e the expected proportion under random assignment. Values of $\kappa > 0.80$ indicate strong agreement (Landis and Koch, 1977).

Results. Table 2 reports the outcome of this review.

Table 2: Expert quality control results on the 279-image sample.

Metric	Value
Images reviewed	279
Observed agreement (p_o)	0.94
Cohen’s κ	0.92

The Kappa score of 0.92 indicates strong agreement between the three-annotator consensus and the independent expert, supporting the reliability of the labelled dataset for downstream classification tasks.

4. Dataset Analysis

4.1. Dataset Statistics

The dataset consists of 1,861 images distributed across five disease categories and healthy leaves. It then was randomly split into training (70%), validation (15%), and test (15%) sets while preserving class proportions via stratified sampling. Table 3 summarizes the overall statistics.

Table 3: Summary statistics of the AvocadoLeaf dataset.

Metric	Value
Total images	1,861
Train images	1,302
Validation images	279
Test images	280
Number of classes	6
Min images per class	165
Max images per class	654
Imbalance ratio (max/min)	3.96

The class distribution is shown in Figure 4. The most frequent class is *domla* (654 images), while the least frequent is *repsap* (165 images). The imbalance ratio of

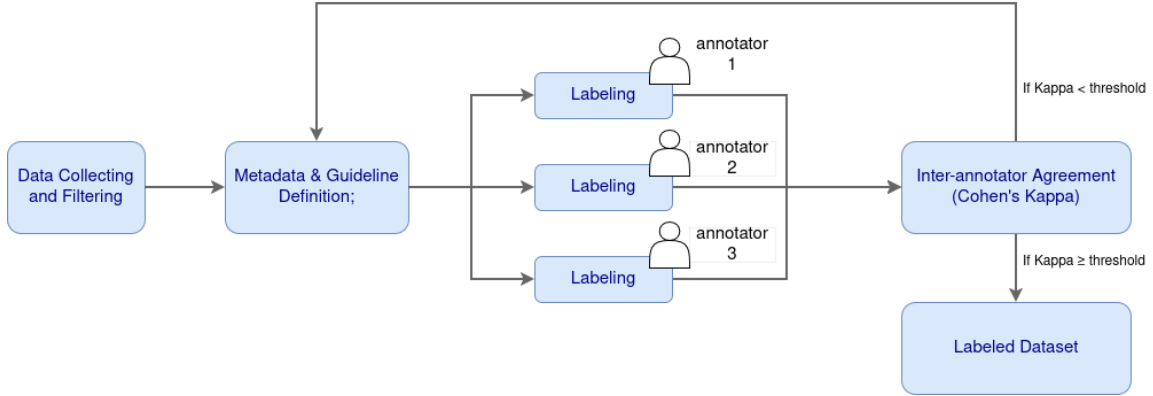


Figure 3: Overview of the data annotation pipeline, including the re-labelling loop described below.

3.96 indicates moderate class imbalance. Transfer learning (Section 5) helps this sample size go further than training from scratch would allow. Section 7 revisits how the resulting class sizes, particularly for *repsap* and *gisat*, still bound how confidently the rarest classes’ results should be read. The per-split distribution is also displayed, confirming that the stratification was effective.

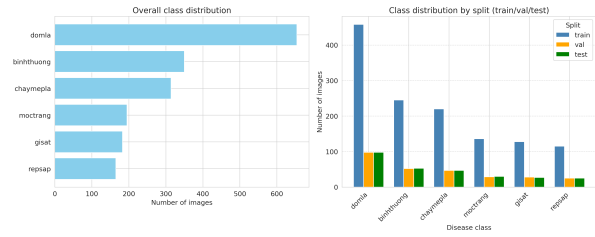


Figure 4: Class distribution. Left, overall bar chart. Right, grouped bar chart showing train/validation/test splits.

4.2. Photometric Analysis

As supporting evidence for the imaging diversity claimed in Section 3, Figure 5 shows that brightness varies substantially even within a single class (standard deviation 9.0 to 24.9 across classes), with *domla* images alone spanning 44.4 to 153.9 and *gisat* images spanning 80.5 to 182.1 on the 0-255 scale. Figure 6 shows that classes also differ in dominant colour channel. These variations confirm that the images were collected under diverse, uncontrolled lighting and colour conditions rather than a single studio-like setup, and imply that models must be robust to such shifts rather than exploit them as shortcuts.

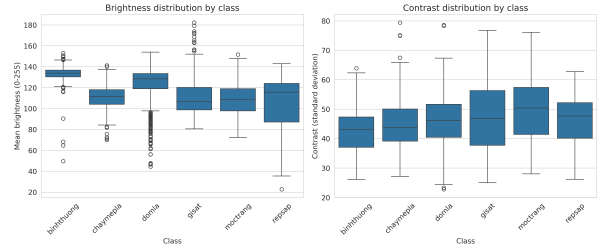


Figure 5: Brightness (left) and contrast (right) distributions per class.

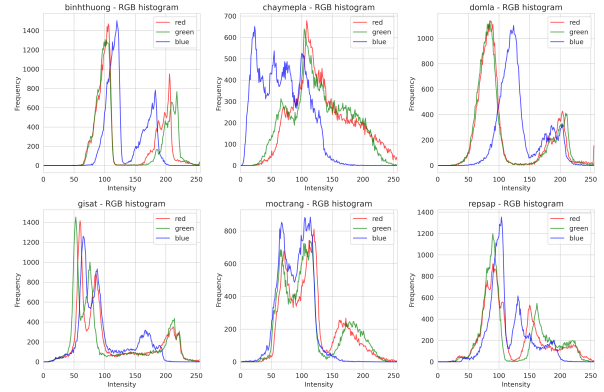


Figure 6: RGB histograms of one sample image per class (red, green, blue channels).

4.3. Intrinsic Difficulty Analysis

To assess how challenging the dataset is for automatic classification, we performed three complementary analyses on the validation set using features extracted from a frozen ImageNet-pretrained ResNet-18 (He et al., 2016). The feature dimension was reduced to 512 via global average pooling.

4.3.1. Quantitative Clustering Metrics

We computed two standard metrics that evaluate class separability in the feature space, namely the Silhouette score (Rousseeuw, 1987) and the Davies-Bouldin index (Davies and Bouldin, 1979). Table 4 summarises the results.

Table 4: Quantitative difficulty metrics on the validation set. Lower Silhouette, higher Davies-Bouldin, and intra/inter ratio ≥ 1 indicate strong class overlap.

Metric	Value
Silhouette Score	0.0067
Davies-Bouldin Index	4.17
Average intra-class distance	12.82
Average inter-class distance	8.04
Intra/Inter distance ratio	1.59

The Silhouette score near zero indicates that samples are not closer to their own class than to neighbouring classes, implying heavy inter-class overlap. The Davies-Bouldin index (4.17) points in the same direction (lower is better separated, metric definition in Appendix B), though its absolute value is best read relative to other clustering configurations rather than against a fixed threshold, so it is reported here as a corroborating signal alongside Silhouette and the intra/inter distance ratio below. Notably, the intra/inter distance ratio exceeds 1 (1.59), meaning that within-class variation is larger than the separation between class centroids, a hallmark of fine-grained recognition problems.

4.3.2. Visualisation with t-SNE

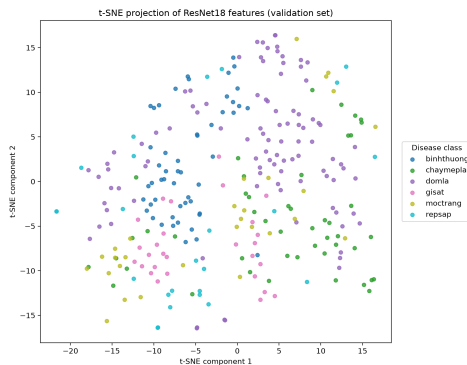


Figure 7: t-SNE projection of ResNet-18 features on the validation set. Overlapping clusters confirm that disease categories are not well separated in the deep feature space.

We projected the 512-dimensional features into two dimensions using t-SNE (van der Maaten and Hinton,

2008) with perplexity 30. Figure 7 shows the resulting scatter plot, where each point represents a validation image and colours encode disease classes. All six classes exhibit substantial overlap without clear cluster boundaries. Classes with visually similar symptoms, such as *chaymepla* (leaf scorch) and *repsap* (mealybug damage), are heavily interleaved. This visual entanglement further confirms that the disease categories are not well separated in the deep feature space.

4.3.3. Linear Separability Test

To determine whether the frozen deep features are linearly separable by disease class, we trained a logistic regression classifier on the extracted features. Using 5-fold cross-validation, the model achieved a mean accuracy of $75.25\% \pm 3.76\%$. When evaluated on a held-out subset (30% of the validation set, disjoint from the folds used above), the accuracy dropped to 65.48%. Part of this gap is likely attributable to the small size of the validation set (279 images), which inflates the variance of both estimates. This attribution is qualitative. Unlike the architecture comparison in Section 6 (Appendix C), we did not compute a bootstrap confidence interval for the held-out estimate. The size of the reported drop should therefore be read as a point estimate whose own precision is not quantified here. Nonetheless, both figures are significantly better than random guessing (16.7% for six classes). For comparison, Section 6 shows that a fine-tuned, non-linear ConvNeXt-V2 model reaches 92.14% on this same AvocadoLeaf test set, a gap of roughly 17-27 percentage points over the linear probe depending on which estimate (75.25% or 65.48%) is used. Hence, the disease classes are **not linearly separable** in the feature space, requiring non-linear models or fine-tuning to achieve satisfactory accuracy.

Taken together, the three analyses in this section are consistent with one another. Near-zero Silhouette, high Davies-Bouldin, and an intra/inter distance ratio above 1 all indicate substantial class overlap in feature space. The t-SNE visualisation shows this overlap directly. The linear probe confirms that this overlap cannot be resolved by a linear decision boundary. This motivates both the comparison of non-linear, fine-tuned architectures in Section 6 and, more specifically, the use of metric-based few-shot learning in Section 6.4. Since frozen features are not linearly separable, a classifier trained directly on top of them generalises poorly, whereas prototypical networks learn a task-adapted embedding space in which the classes become separable.

5. Proposed Lightweight Model

Rather than designing a new architecture from scratch, we adopt a systematic *selection-and-adaptation* approach. A broad pool of existing lightweight and efficient architectures is fine-tuned under a largely shared training protocol, with the deviations documented below, on the AvocadoLeaf dataset, and the best-performing candidate under joint accuracy-size-latency constraints is proposed as the deployment model for this task (Section 6). This section describes the candidate pool, the transfer-learning strategy applied to each, and the shared training protocol.

5.1. Candidate Architectures

We evaluate 15 architectures spanning five groups (a finer split of the three broader CNN / hybrid CNN-Transformer / pure-transformer families referenced in Section 2):

- (1) a Vanilla CNN trained from scratch as a minimal baseline.
- (2) a ResNet-18 (He et al., 2016) baseline.
- (3) mobile-oriented CNNs (MobileNet-V2 (Sandler et al., 2018), MobileNet-V4 (Qin et al., 2024), TinyNet (Han et al., 2020), GhostNet-V2 (Tang et al., 2022), EfficientNet-B0 (Tan and Le, 2019)).
- (4) hybrid CNN-Transformer designs (MobileOne (Vasu et al., 2023b), EdgeNeXt (Maaz et al., 2022), FastViT (Vasu et al., 2023a), EfficientViT (Liu et al., 2023), ConvNeXt-V1 (Liu et al., 2022), ConvNeXt-V2 (Woo et al., 2023)).
- (5) pure vision transformers (ViT-B/16 (Dosovitskiy et al., 2021), SwinTransformer (Liu et al., 2021)).

All architectures except Vanilla CNN are initialised from ImageNet-pretrained weights obtained from `torchvision` or `timm`.

5.2. Transfer-Learning Strategy

Because the AvocadoLeaf training set is comparatively small (1,302 training images across six classes), fully fine-tuning all layers of large pretrained backbones risks overfitting. We therefore apply *selective layer unfreezing*. For every pretrained model, all backbone weights are frozen except the last one to two stages (or the last two encoder layers, for the transformer-based models) and a newly initialised classification head sized to the six AvocadoLeaf classes. Table 5 summarises the unfreezing strategy per architecture family. The Vanilla CNN baseline has no pretrained weights and is instead trained end-to-end from random initialisation.

5.3. Training Protocol

All 15 models share the same optimiser, learning-rate schedule, epoch budget, and data augmentation policy, using AdamW (learning rate 1×10^{-4} , weight decay 1×10^{-4}) with cosine-annealing decay over a maximum of 100 epochs, cross-entropy loss, and early stopping with a patience of 5 epochs on validation loss. Input images are resized to 224×224 and normalised using ImageNet statistics. Training-time data augmentation consists of random-resized cropping (scale 0.6–1.0), horizontal flipping, random rotation ($\pm 15^\circ$, applied with probability 0.3), colour jittering (brightness/contrast/saturation/hue), and random erasing. Models are always evaluated without augmentation at validation and test time.

Two aspects of this protocol are not held perfectly constant across all 15 candidates. First, the unfrozen-layer depth in Table 5 differs between ResNet-18 (one stage) and the remaining pretrained models (two stages or encoder layers). Second, despite the augmentation policy above being intended uniformly, the ConvNeXt-V2/BoVien checkpoint reported in Section 6 was in fact trained with augmentation disabled, and this switch was not applied consistently across all candidates. The ablation study (Section 6.1.2) examines its effect on BoVien directly. See the Limitations discussion (Section 7) for further detail on both points.

The architecture that achieves the best joint accuracy-size-latency trade-off under this protocol, ConvNeXt-V2, is adopted as our proposed lightweight model for avocado disease classification. The full comparison against the other 14 candidates is reported in Section 6.

5.4. Proposed Model Architecture

As introduced in Section 1, we name the resulting deployment model **BoVien**, the `convnextv2_atto` variant of ConvNeXt-V2 (3.7M parameters with its original 1000-way ImageNet head, 0.55 GFLOPs at 224×224), the smallest in the ConvNeXt-V2 family, adapted to AvocadoLeaf via the selective layer unfreezing strategy of Table 5. Replacing this head with the six-way AvocadoLeaf classification head used throughout this paper brings the parameter count down to the 3.4M reported in Table 6. Figure 8 details its architecture. Box width encodes channel count, and box height/depth encode spatial resolution. Each stage’s internals are grouped into a single box instead of listing every submodule. Reading left to right, a $4 \times 4/\text{stride-4}$ convolutional stem produces a $56 \times 56 \times 40$ feature map, which then passes through four stages of ConvNeXt-V2 blocks (depths 2-2-6-2, widths 40-80-160-320) with 2×2 downsampling

Table 5: Transfer-learning strategy by architecture family.

Architecture family	Members	Unfrozen layers
Vanilla CNN	–	All (trained from scratch)
ResNet-18	–	layer4 + classifier head
Mobile CNNs	MobileNet-V2, MobileNet-V4, TinyNet, GhostNet-V2, EfficientNet-B0	Last 2 feature blocks + head
Hybrid CNN-Transformer	MobileOne, EdgeNeXt, FastViT, EfficientViT, ConvNeXt-V1, ConvNeXt-V2	Last 2 stages + head
Vision Transformers	ViT-B/16, SwinTransformer	Last 2 encoder layers + head

between stages, ending at $7 \times 7 \times 320$. This is visible in the figure as boxes that shrink in height/depth (spatial resolution) while growing in length (channel count). Following Table 5, Stages 1-2 (grey) remain frozen at their ImageNet-pretrained weights, while Stages 3-4 (orange) are fine-tuned on AvocadoLeaf. Each stage is a stack of ConvNeXt-V2 blocks, each consisting of a 7×7 depth-wise convolution, LayerNorm, an inverted-bottleneck MLP ($C \rightarrow 4C \rightarrow C$) with GELU activation, and Global Response Normalisation (GRN) inserted between the two pointwise layers (the modification that distinguishes ConvNeXt-V2 from ConvNeXt-V1’s LayerScale-based block), the whole block wrapped in a residual connection. The classification head (global average pool $\rightarrow$ LayerNorm $\rightarrow$ linear) is entirely new (green) and pools the 320-d backbone feature, mapping it to a probability over the six AvocadoLeaf classes, kept deliberately narrow in the figure since it carries only 6 output channels. The leftmost image is a real (background-removed) AvocadoLeaf training sample, shown at its actual $224 \times 224 \times 3$ input resolution. In the remainder of the paper, “ConvNeXt-V2” refers to this candidate within the 15-architecture comparison (Section 6), and “BoVie” refers specifically to it once selected as the final proposed model.

6. Experiments and Results

6.1. Comparison of Architectures

We benchmark a wide range of lightweight and efficient architectures on the AvocadoLeaf test set. Table 6 reports top-1 accuracy, macro F1 score, number of parameters, and per-image inference time on a single NVIDIA RTX 3050-6GB GPU for all 15 evaluated models. All models share the same optimiser, learning-rate schedule, and data augmentation policy (Section 5). Each architecture was trained once (single seed). To account for the uncertainty this and the finite 280-image test set introduce, we additionally report bootstrap 95%

confidence intervals and paired significance tests in Appendix C.

Among the evaluated models, ConvNeXt-V2 achieves the highest point-estimate classification accuracy (92.14%) and the second-highest macro F1 score (0.900), while maintaining a compact parameter count (3.4M) and low inference latency (2.30 ms). However, bootstrap significance testing (Appendix C) shows that its accuracy advantage over the next five models (SwinTransformer, MobileOne, ViT, EdgeNeXt, and ConvNeXt-V1) is *not* statistically significant at this test set size ($p > 0.5$ for all five). These six models should be regarded as statistically tied in accuracy. In particular, ViT attains the highest macro F1 (0.904) but has 85.8M parameters and requires 14.25 ms per inference, making it impractical for edge deployment despite its statistically indistinguishable accuracy. What separates ConvNeXt-V2 from the rest of this tied top group is not proven superior accuracy but a far smaller parameter count (3.4M vs. 4.3–85.8M for the other five) and lower latency, which is why we select it as our proposed lightweight model. ConvNeXt-V2’s advantage over every model ranked seventh or lower *is* statistically significant ($p < 0.05$, Appendix C). Conversely, lightweight architectures such as MobileNet-V2 and EfficientNet-B0 are faster and smaller but lag the top-performing group by approximately 5–6 percentage points (approximately 86–87%). Models with accuracy below 80% (MobileNet-V4, EfficientViT, GhostNet-V2, VanillaCNN) perform substantially worse under the present training protocol. Overall, ConvNeXt-V2 offers the most favourable balance among accuracy, model size, and speed, making it the strongest candidate among the evaluated architectures for deployment on resource-constrained devices, an assessment corroborated by the on-device mobile deployment results in Section 6.7.

To further understand the impact of model capacity, we group the 14 fine-tuned, ImageNet-pretrained architectures into two categories, including (1) *lightweight*

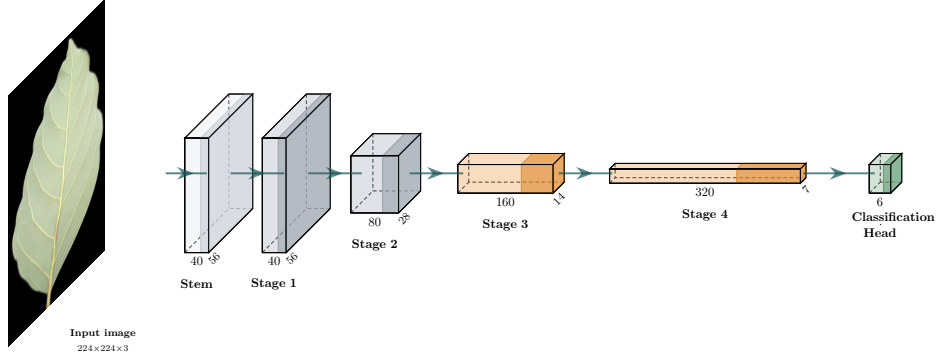


Figure 8: Architecture of Model BoVien. Grey indicates frozen (ImageNet-pretrained, unchanged) parameters, orange indicates parameters fine-tuned on AvocadoLeaf, and green indicates newly initialised, always-trainable parameters.

Table 6: Performance comparison of all evaluated architectures. The highest values in each column are highlighted in bold and underlined.

Model	Acc (%)	Macro F1	Params (M)	Inference (ms)
ConvNeXt-V2	<u>92.14</u>	0.900	3.4	2.30
SwinTransformer	91.43	0.899	27.5	6.69
MobileOne	91.43	0.893	4.3	8.35
ConvNeXt-V1	91.07	0.895	27.8	5.24
ViT	91.07	<u>0.904</u>	85.8	14.25
EdgeNeXt	91.07	0.877	5.3	3.75
ResNet-18	88.93	0.863	11.2	1.98
TinyNet	88.21	0.863	4.9	3.66
FastViT	87.50	0.853	3.3	3.63
MobileNet-V2	87.14	0.847	2.2	1.80
EfficientNet-B0	86.43	0.838	4.0	2.85
MobileNet-V4	76.07	0.724	2.5	1.89
EfficientViT	69.29	0.650	3.2	7.00
GhostNet-V2	69.29	0.571	4.9	5.43
VanillaCNN	63.57	0.623	25.8	1.33

models with fewer than 5 million parameters (9 models) and (2) *heavy models* with 5 million or more (5 models). The VanillaCNN baseline is excluded from this comparison: its 25.8M parameter count reflects an unoptimised from-scratch architecture rather than a deliberate heavy-model design choice, and including it by parameter count alone would place a 0.623 F1 score inside the heavy group purely as an artefact of training regime, not capacity. The heavy group achieves a high and stable mean F1 of 0.888, with all models scoring above 0.86. The light group shows much greater variance, with a mean of 0.793 and scores ranging from 0.571 to 0.900. ConvNeXt-V2 stands out as the only light model attaining F1 (0.900) comparable to the best heavy architectures while using substantially fewer parameters. This confirms that model selection is critical under parameter constraints, whereas heavy models offer stable per-

formance at the cost of larger size and slower inference.

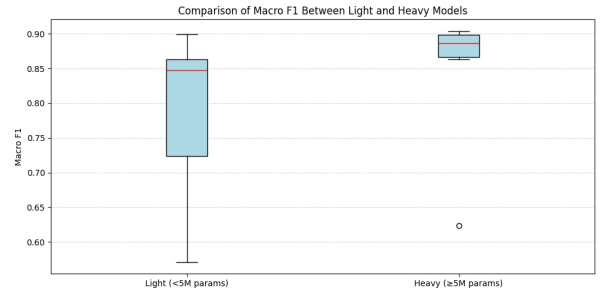


Figure 9: Macro F1 scores of all evaluated models. Lightweight models (<5M parameters) are shown in blue, heavy models (≥5M) in orange.

6.1.1. Efficiency Analysis

To jointly assess accuracy, model size, and inference speed, we define a study-specific composite efficiency score:

$$\text{Efficiency} = \frac{\text{Macro F1}}{\text{Params (M)} \times \text{Inference Time (ms)}}.$$

This metric quantifies the predictive performance achieved per unit of resource consumption, where higher values indicate better trade-offs among the three dimensions. Figure 10 visualises two complementary efficiency dimensions, F1 per million parameters (F1/Params) and F1 per millisecond inference time (F1/Time). The colour scale indicates macro F1.

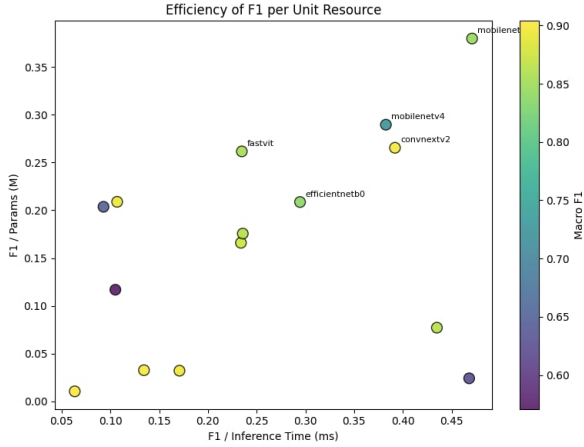


Figure 10: Scatter plot of F1 per parameter versus F1 per inference time. Colour indicates macro F1.

MobileNet-V2 achieves the highest overall efficiency (0.214) due to its extremely compact size (2.2M parameters) and fast inference (1.80 ms), though its macro F1 (0.847) is moderate. ConvNeXt-V2 ranks third in efficiency (0.115) while attaining a substantially higher F1 (0.900). Its F1/Params (0.265) and F1/Time (0.391) are well balanced, placing it in the upper-right region of the scatter plot. Heavy models such as ViT (0.001) and SwinTransformer (0.005) exhibit very low efficiency despite competitive F1, due to large parameter counts and slower inference. These results indicate that ConvNeXt-V2 offers the most favourable balance between accuracy and resource utilisation among the evaluated architectures, based on parameter count and GPU inference latency.

6.1.2. Ablation Study

To isolate the contribution of two training-protocol switches exposed by our pipeline, namely data augmen-

tation and class-weighted loss, we retrain Model BoVien (ConvNeXt-V2) under all four combinations, holding the architecture, optimiser, schedule, and epoch budget fixed. This does not repeat the full 15-architecture sweep. It only probes the proposed model itself. Each configuration is again trained once (single seed) on the same 280-image test set, so, following the same protocol as Section 6.1, we report bootstrap 95% confidence intervals and paired significance tests in Appendix C.1.

Table 7: Ablation of augmentation and class-weighted loss on Model BoVien (AvocadoLeaf test set).

Configuration	Acc (%)	Macro F1
No augmentation, no class-weight	92.14	0.900
No augmentation, class-weighted	90.71	0.887
Augmentation, no class-weight	88.93	0.864
Augmentation, class-weighted	91.43	0.895

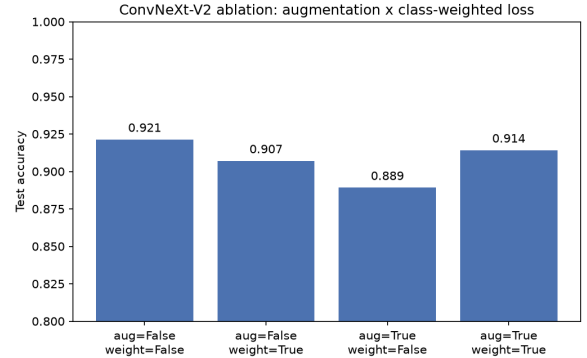


Figure 11: Test accuracy of Model BoVien under the four augmentation $\times$ class-weighted-loss combinations.

The no-augmentation, no-class-weight configuration is the best of the four, and its accuracy (92.14%) matches the ConvNeXt-V2 row of Table 6 exactly. This ablation grid includes the very checkpoint reported as our proposed model, not a separately retrained approximation of it, and confirms that the reported BoVien checkpoint was trained *without* augmentation. This differs from the augmentation policy described in Section 5, which we flag as a documentation/protocol discrepancy in Section 7 rather than silently reconcile here. Adding augmentation alone costs 3.2 accuracy points (92.14% $\rightarrow$ 88.93%), the largest single-factor drop in the grid, and adding class-weighting on top of augmentation recovers about two-thirds of that loss (88.93% $\rightarrow$ 91.43%) but still falls short of the no-augmentation baseline. Class-

weighting alone also underperforms the plain baseline, though by a smaller margin (92.14% $\rightarrow$ 90.71%). A plausible explanation for augmentation hurting this particular model is that, on AvocadoLeaf’s comparatively small training set (1,302 images), the added geometric and photometric perturbations remove information that ConvNeXt-V2’s frozen-backbone-plus-fine-tuned-stages setup would otherwise use productively, though this ablation grid alone cannot rule out that the same holds for the other 14 candidate architectures. However, bootstrap significance testing (Appendix C.1) shows that none of these three differences are statistically significant at this test set size, including the largest one (no augmentation, no class-weight, versus augmentation, no class-weight, +3.31pp, $p = 0.054$). The ranking above should therefore be read as suggestive rather than conclusive, consistent with how Section 6.1’s single-seed, 280-image comparisons are treated throughout this paper.

6.1.3. Error Analysis

To characterise *which* classes drive the remaining test-set errors, rather than only the aggregate 92.14% figure, we re-examine BoVien’s 280 per-image test predictions (already saved by the evaluation pipeline, no re-inference) through its confusion matrix.

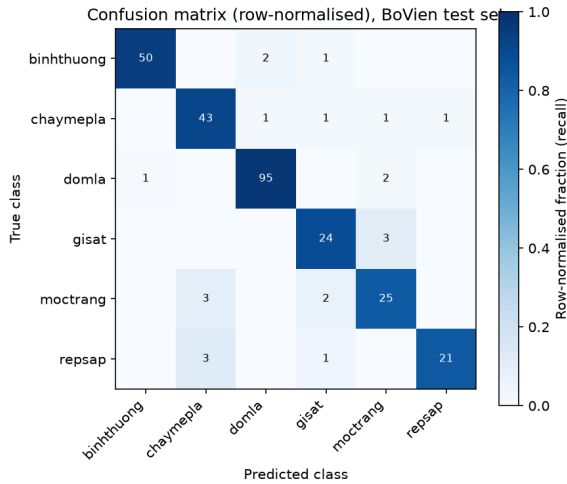


Figure 12: Row-normalised confusion matrix (raw counts annotated) for Model BoVien on the AvocadoLeaf test set.

Per-class recall ranges from 96.9% (*domla*, the largest class at 458 training images) down to 83.3% (*moctrang*, 136 training images). The three smaller classes, *repsap* (115 training images, the smallest, 84.0% recall), *gisat* (128, 88.9%), and *moctrang*, all fall short of the

91%+ recall reached by the three larger classes (*domla* 458, *binhthuong* 245, *chaymepla* 220), broadly consistent with a sample-size effect. *Moctrang* is nonetheless a partial exception to that pattern: despite having more training images than both *repsap* and *gisat*, it has the lowest recall of the six classes, suggesting that beyond sample size, its diffuse white-powdery symptom pattern (frequently confused with *chaymepla*, discussed below) is intrinsically harder to classify. The confusions are concentrated in three symmetric pairs: *gisat* (algal leaf spot, orange velvety pustules) $\leftrightarrow$ *moctrang* (powdery mildew, white powdery coating) account for 5 of 22 misclassified test images in both directions, *repsap* (mealybug damage, silver/stippled marks) is confused with *chaymepla* (leaf scorch, margin/tip burn) in 12.0% of *repsap* images, and *moctrang* is likewise confused with *chaymepla* in 10.0% of its images. All three pairs share a plausible visual explanation: each involves a diffuse or patchy discolouration (algal spots, powdery coating, insect stippling) that can visually resemble scorch damage or a different spot pattern under variable field lighting, rather than the compact, high-contrast lesions of *domla*. Notably, *repsap* is also the class with the lowest LIME faithfulness score (Section 6.2, mean confidence drop 0.416 vs. 0.649 overall), so its diffuse, non-contiguous damage pattern appears to make it both harder to classify correctly and harder to explain with a fixed-size superpixel mask, two independent pieces of evidence pointing at the same underlying cause.

6.2. Model Interpretability

To assess whether ConvNeXt-V2 relies on disease-relevant visual evidence rather than spurious correlations, we apply LIME (Ribeiro et al., 2016) to every image in the 280-image test set (not merely a qualitative sample). Each image is segmented into superpixels with the Quickshift algorithm, and 500 perturbed samples are used to fit a local surrogate model that identifies the superpixels most responsible for the predicted class.

We quantify faithfulness with a deletion metric. For each test image, we hide exactly the superpixels LIME identifies as most responsible for the predicted class (the same masking colour used during LIME’s own perturbation sampling) and re-run the model, measuring how much its confidence in the predicted class drops. A faithful explanation should cause a large drop, since it claims the hidden region is what the prediction relies on. Averaged over all 280 test images, hiding the LIME-highlighted region (which covers only 20.0% of the image area on average) drops the model’s confidence in its own predicted class from 0.940 to 0.292, a mean confidence drop of 0.649 (bootstrap 95% CI

[0.618, 0.678], see Appendix D for the full methodology and per-class breakdown), demonstrating that predictions rely strongly on the LIME-attributed regions. The drop is larger for correctly classified images (0.660, $n = 258$) than for misclassified ones (0.516, $n = 22$), qualitatively, these highlighted regions visually align with disease-associated symptoms rather than background noise. This full-test-set quantitative result substantially strengthens the qualitative examples below.

Figure 13 shows two representative qualitative examples. For *chaymepla*, the highlighted superpixels tightly enclose the brown, scorched lesion at the centre of the leaf, closely matching the visible symptom boundary. For *moctrang* (powdery mildew), the highlighted regions concentrate on the white powdery patches on the leaf surface rather than on the unaffected tissue or background. These qualitative results illustrate that ConvNeXt-V2’s predictions are driven by the same visual cues a human expert would use, rather than by background or imaging artefacts, consistent with the quantitative deletion-metric result above.



Figure 13: LIME explanations for ConvNeXt-V2 on two disease classes. Left, *chaymepla* (leaf scorch), where the highlighted region matches the scorched lesion. Right, *moctrang* (powdery mildew), where the highlighted region matches the white powdery patches. Yellow outlines mark superpixels positively attributed to the predicted class.

6.3. Multi-modal Explanation Generation (CNN + LIME + VLM)

LIME identifies *which pixels* a prediction depends on but produces no natural-language account of *why* those pixels are diagnostically relevant, a gap that matters for a farmer-facing tool, where a highlighted region without a caption is of limited practical use. We therefore built a pipeline combining all three components developed in this work (the CNN’s prediction, LIME’s visual attribution, and a small, locally-hosted vision-language model, Qwen2-VL-2B-Instruct, no API cost) to generate a grounded natural-language explanation. Critically, the CNN remains the sole decision-maker. The VLM is given the CNN’s already-decided class label together with the raw leaf image and the LIME overlay image, and is asked only to describe the visual evidence and judge whether it supports that label, never to propose a different one (`final_pred` is verified to equal the

CNN’s prediction for 100% of processed images). This design was applied to the 140 test images ConvNeXt-V2 was least confident about (top 50% by MC-Dropout predictive entropy).

Figure 14 shows three outputs of this pipeline, hand-selected from the 140-image escalated subset as coherent, correctly-grounded examples rather than a random or representative sample (full text in Table 8). In these examples, the generated text correctly references both the LIME-highlighted location (e.g. “vùng giữa hai lá”) and the specific visual symptom expected for the class (e.g. rust-coloured spots for *gisat*, white powdery patches for *moctrang*), demonstrating that the pipeline *can* turn a raw prediction and an attribution map into an interpretable, farmer-readable statement, which was the original motivation for adding a language model to this system. Not every generated explanation is this coherent, however. Section 7 discusses a case where the model’s reasoning conflated the symptom description of a different class.

Across the full 140-image escalated subset, the VLM judged the visual evidence to support the CNN’s diagnosis for 92.0% of the 125 images that produced a parseable response. This figure should not be read as a validated accuracy or quality score. It is simply how often the model answered “yes”, and it is in fact somewhat higher than ConvNeXt-V2’s own accuracy on either grouping, 84.8% on the same 125-image parseable subset or 85.0% on the full 140-image escalated set (Appendix E). This is consistent with the low recall (0.21) for detecting actual mistakes reported in Section 7, as the model agrees with the CNN more often than the CNN is actually correct, suggesting a degree of acquiescence bias rather than critical evaluation of the evidence.

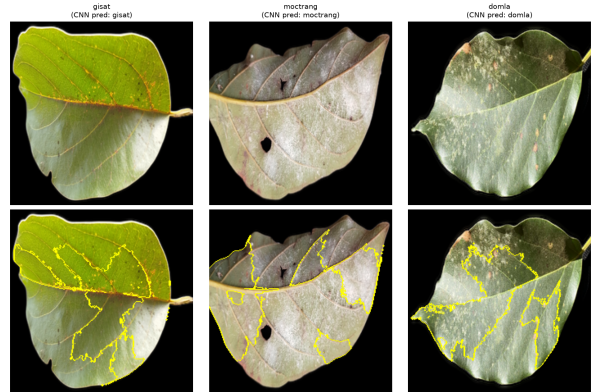


Figure 14: Three examples of the combined CNN + LIME + VLM explanation pipeline. Top row, original test images. Bottom row, LIME overlay (yellow outlines) shown to the VLM alongside the CNN’s prediction. Generated explanations are given in Table 8.

Table 8: VLM-generated explanations for the three examples in Figure 14, translated from Vietnamese.

Class	Generated visual evidence
gisat	“The CNN’s most-attended region is in the middle of the leaf, particularly the area with rust-orange-coloured spots, typically on the underside – this may be a sign of <i>gisat</i> .”
moctrang	“The estimated region between the two images has many yellow-outlined patches, indicating white powdery spots on the leaf – the CNN attended most to this region when predicting <i>moctrang</i> .”
domla	“The CNN’s most-attended region is near the centre of the leaf, close to the leaf-spot location; the spot’s colour and shape resemble the small, round or irregular spots on the leaf surface [that define <i>domla</i>].”

This capability is distinct from, and should not be conflated with, using the same VLM output as an automatic error detector. Section 7 and Appendix E report a separate, secondary analysis of whether the `supports_cnn_diagnosis` judgement can substitute for or complement predictive-entropy-based flagging of likely CNN mistakes, which was *not* successful. The explanation-generation capability demonstrated here is unaffected by that negative result. The pipeline is *capable* of producing a plausible, grounded caption for a prediction, as shown by the curated examples above, though we have not established how often it does so unprompted across the full subset, nor validated the captions against expert judgement rather than an automated proxy. Whether such captions are a reliable enough signal to automatically flag errors without human review is a separate, harder question that Section 7 answers in the negative for this specific use.

6.4. Few-shot Learning

To evaluate the ability of our model to generalise from very few labelled examples, a realistic scenario in agricultural applications where data collection is expensive, we conducted few-shot experiments on the AvocadoLeaf dataset. We compared three approaches, namely (i) standard fine-tuning of the entire classifier on the limited support set (denoted as `Fine-tune`), (ii) Prototypical Networks (ProtoNet) using a frozen ConvNeXt-V2 backbone, and (iii) Relation Networks (RelationNet) with the same frozen backbone.

For each setting, we randomly sampled K images per class ($K \in \{1, 5, 10, 20\}$) from the training set to form the support set, and evaluated on the full test set.

Fine-tune was trained for 50 epochs with early stopping. For ProtoNet and RelationNet we report the mean accuracy and standard deviation over 600 randomly generated episodes (6-way, K -shot, 5 query images per class), drawn without a fixed random seed, so a repeated run reproduces the qualitative ranking below but not the exact reported figures within roughly one standard deviation.

The quantitative results are summarised in Table 9, and a visual comparison is provided in Figure 15. Several observations can be made:

- **Fine-tune suffers from severe overfitting** when only a few samples per class are available. With only 5 shots, it achieves only 16.4% accuracy, and even with 20 shots it reaches only 28.6%, indicating that conventional fine-tuning is not suitable for extreme low-data regimes.
- **Prototypical Networks substantially outperforms fine-tuning at all shot levels.** Already with 1 shot per class, ProtoNet attains 28.9% accuracy, which is higher than fine-tuning with 5 shots (16.4%). The gap remains large as the number of shots increases. At 5 shots ProtoNet achieves 45.7% (+29.2 p.p.), at 10 shots 52.3% (+31.3 p.p.), and at 20 shots 57.4% (+28.9 p.p.). These results demonstrate that metric-based few-shot learning is substantially more effective than fine-tuning for fine-grained plant disease classification, even though absolute accuracy remains moderate.
- **Relation Networks perform poorly** on this dataset, with accuracy remaining close to the 16.7% random-chance baseline (16–19%) across all shot levels and showing no consistent improvement as the number of shots increases (see Figure 15). This suggests that the learnable distance metric of RelationNet may overfit to the specific episode composition or that the relation module requires more training data to generalise.

Overall, the ProtoNet framework combined with a frozen ConvNeXt-V2 backbone provides the strongest few-shot baseline among the three methods evaluated, more than doubling fine-tuning’s accuracy at every directly comparable shot level (5, 10, and 20 shots) and reaching 57.4% accuracy with only 20 labelled images per class. While this is well below the >90% accuracy achieved with the full training set (Table 6), it demonstrates that metric-based few-shot learning is a substantially more data-efficient starting point than fine-tuning for scenarios where data collection is expensive.

Table 9: Few-shot classification accuracy (%) on the test set. Fine-tune results are from a single training run. ProtoNet and RelationNet results are averaged over 600 random episodes (mean $\pm$ std).

Method	1-shot	5-shot	10-shot	20-shot
Fine-tune	–	16.43	21.07	28.57
ProtoNet	28.9 $\pm$ 8.3	45.7 $\pm$ 9.1	52.3 $\pm$ 8.8	57.4 $\pm$ 9.0
RelationNet	16.2 $\pm$ 5.9	18.5 $\pm$ 6.3	17.5 $\pm$ 6.2	19.4 $\pm$ 6.9

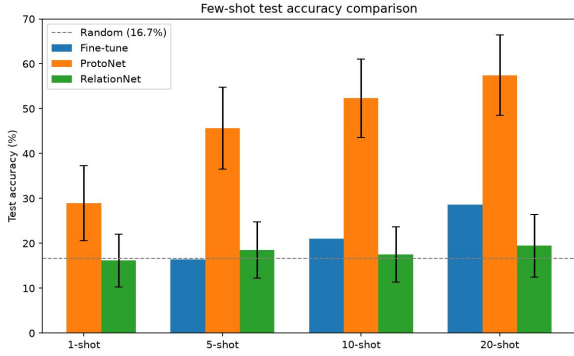


Figure 15: Few-shot test accuracy comparison. Error bars represent standard deviation (ProtoNet, RelationNet). Fine-tune results are single runs.

6.5. Uncertainty Estimation

A high accuracy alone does not guarantee that a model knows when it is unsure. We therefore examine two complementary aspects, calibration and out-of-distribution detection.

6.5.1. Calibration

We compute the Expected Calibration Error (ECE) on the validation set. Our baseline softmax model achieves an ECE of 0.0625. This is somewhat above the common threshold of 0.05 used to indicate good calibration, suggesting the model’s raw softmax confidence is only moderately well calibrated and would benefit from post-hoc calibration before its confidence scores are used directly in high-stakes decisions.

We apply single-parameter temperature scaling (Guo et al., 2017): a scalar T is fit by minimising negative log-likelihood on the validation logits, then used to rescale logits before softmax at inference (T does not change arg-max predictions, so accuracy is unaffected by construction). The fitted temperature is $T = 1.274$ ($T > 1$ indicates the raw model is overconfident, consistent with the ECE above 0.05). On the validation set this reduces ECE from 0.0625 to 0.0505, essentially at the 0.05 threshold. On the held-out test set, independent of the temperature fit, ECE improves from 0.0311 to 0.0193,

clearly within the good-calibration range. Figure 16 shows the corresponding reliability diagrams. Temperature scaling is a one-parameter, post-training operation with no retraining and negligible inference-time cost, so we recommend it be applied whenever BoVien’s raw confidence is surfaced to an end user.

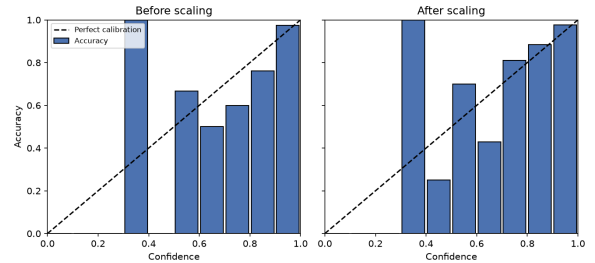


Figure 16: Reliability diagrams (test set, 10 bins) before and after temperature scaling ($T = 1.274$). Bars show empirical accuracy per confidence bin. The dashed diagonal is perfect calibration. Some bins are sparsely populated given the 280-image test set, which is why individual bars can look noisy even as the aggregate ECE improves.

6.5.2. Far out of distribution detection

We collect 200 random images from the internet, including animals, landscapes and objects. These images are completely unrelated to leaves. Using Monte Carlo Dropout with 50 forward passes, we compute the predictive entropy. The area under the ROC curve reaches 0.994, meaning the model almost perfectly separates avocado leaf images from far domain images. The mean entropy for in distribution validation samples is 0.21, while for far OOD samples it is 1.46. This large gap confirms that the model becomes highly uncertain when confronted with entirely irrelevant inputs.

Figure 17 shows the entropy histogram for this experiment.

6.5.3. Near out of distribution detection

A more realistic challenge comes from leaf images of other crop species. We take 200 images from the PlantVillage dataset, containing leaves of tomato, corn and potato. These images still depict leaves but belong

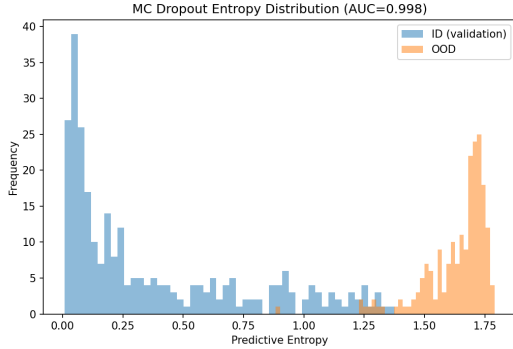


Figure 17: Entropy distribution on far OOD (random internet images). ID samples concentrate at very low entropy, OOD samples at high entropy, with almost no overlap.

to plants that are never seen during training. The model achieves an AUC of 0.985 on this near OOD set, nearly as strong as the far OOD result. The mean entropy for near OOD samples is 1.35, noticeably higher than the 0.21 of the in distribution validation set. This shows that the model’s uncertainty is not merely picking up on trivial domain shift (as in the far OOD case) but also distinguishes avocado leaves from visually similar leaves of other crop species.

Figure 18 presents the corresponding entropy histogram.

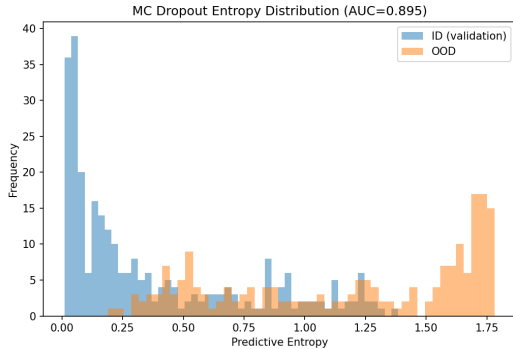


Figure 18: Entropy distribution on near OOD (PlantVillage leaves). ID samples still cluster at low entropy, while near OOD samples shift to high entropy, with only a small overlap, consistent with the AUC of 0.985.

Table 10 summarises the OOD detection results.

6.6. Cross-dataset Evaluation

The uncertainty analysis above shows that ConvNeXt-V2’s predictive entropy reliably *flags* domain-shifted inputs when that signal is explicitly used as a

Table 10: OOD detection AUC using MC Dropout entropy.

OOD set	Type	AUC
Random internet images	Far OOD	0.994
PlantVillage leaves	Near OOD	0.985

detector (Section 6.5). This raises a distinct, practically important question. What would happen if a model were deployed *without* that uncertainty gate, simply taking the arg-max class at face value? This section first quantifies that naive-deployment failure mode on off-task images (Section 6.6.1), then reports, in Section 6.6.2, a genuine same-task train-A/test-B evaluation on an independently collected avocado leaf dataset.

6.6.1. Naive-Deployment Behaviour on Off-Task Inputs

We reuse the existing far-OOD (random internet images) and near-OOD (PlantVillage tomato/corn/potato leaves) sets from Section 6.5 to quantify a complementary, and more immediately actionable, failure mode. For each of three models spanning our accuracy/efficiency trade-off, namely ConvNeXt-V2 (proposed and highest accuracy), MobileNet-V2 (fastest and smallest), and MobileOne (edge-specialised and statistically tied accuracy), we run a single deterministic forward pass over both OOD sets and record the arg-max class and its softmax confidence, with no MC-Dropout averaging and no rejection threshold applied.

The three models diverge sharply. ConvNeXt-V2 is the most conservative, with only 23.5% of far-OOD and 35.5% of near-OOD images assigned confidence above 0.5, and essentially none (0.0% far-OOD, 1.5% near-OOD) exceeding 0.9. MobileNet-V2 is the opposite, with 81.5% of far-OOD and 82.0% of near-OOD images accepted at confidence > 0.5 , and a striking 29.5% (far-OOD) and 22.0% (near-OOD) exceed 0.9 confidence despite depicting no avocado leaf at all. MobileOne sits in between (62.0–71.5% at > 0.5 , 4.0–12.0% at > 0.9). Figure 19 summarises the acceptance rate at the 0.5 threshold for all three models and both OOD sets. Table 11 reports the full breakdown, including which classes the rejected mass is (incorrectly) attributed to.

This result reframes the accuracy/efficiency trade-off of Section 6.1. ConvNeXt-V2’s advantage over MobileNet-V2 and MobileOne is not only higher in-distribution accuracy (92.14% vs. 87.14% and 91.43% respectively) but also substantially safer behaviour under distribution shift, i.e. it degrades towards low-confidence, easily-rejectable predictions rather than confidently misclassifying unrelated inputs. Practi-

Table 11: Naive-deployment behaviour on domain-shifted inputs (no uncertainty gate, single forward pass, arg-max class).

Model	OOD set	Mean conf.	% conf.>0.5 / >0.9
ConvNeXt-V2	Far OOD	0.422	23.5 / 0.0
ConvNeXt-V2	Near OOD	0.462	35.5 / 1.5
MobileNet-V2	Far OOD	0.731	81.5 / 29.5
MobileNet-V2	Near OOD	0.700	82.0 / 22.0
MobileOne	Far OOD	0.654	71.5 / 12.0
MobileOne	Near OOD	0.576	62.0 / 4.0

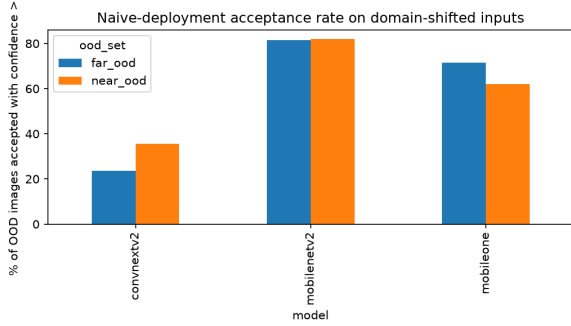


Figure 19: Percentage of out-of-domain images (far-OOD: random internet images, near-OOD: PlantVillage leaves) that a naive arg-max deployment, with no uncertainty gate, would accept at confidence > 0.5, for each of the three demo models.

cally, this means that if a lighter model such as MobileNet-V2 is chosen for a resource-constrained deployment on the strength of its speed and size (Section 6.1.1), the entropy-based uncertainty gate already validated in Section 6.5 is not optional but necessary. Without it, roughly four in five domain-shifted inputs would be silently and confidently misdiagnosed as a specific avocado disease.

6.6.2. Real Cross-Domain Evaluation (Train A, Test B)

Section 6.6.1 characterises behaviour on off-task inputs, but does not answer the question that matters most for external validity: trained only on AvocadoLeaf (domain A, collected first-hand in Vietnam), does BoVien still recognise avocado leaf disease on leaves it has never seen, collected independently, in a different country, under a different camera and background? K-Kotagiri (2024) (Table 1), a 435-image, healthy/diseased-labelled avocado leaf dataset collected in Tamil Nadu, India, lets us test this directly for the first time. Because Kotagiri provides only binary labels, BoVien’s 6-way prediction is collapsed to binary (*binhthuong* → healthy, the other five disease classes → diseased) for this comparison, a same-task but coarser test than the full 6-class problem. To isolate genuine domain shift in leaf and dis-

ease appearance from a simple background-format mismatch, every Kotagiri image is passed through the same background-removal preprocessing (rembg-based foreground segmentation, crop, resize to 224×224) used to build the processed AvocadoLeaf training data itself, rather than fed to the model on its original concrete/stone background.

Table 12: Zero-shot cross-domain evaluation: BoVien trained on AvocadoLeaf (domain A), evaluated with no retraining on both its own test set and K-Kotagiri (domain B), both collapsed to binary (healthy vs. diseased) for a like-for-like comparison.

Metric	Domain A	Domain B
n	280	435
Accuracy	98.57%	60.00%
Precision (dis.)	0.987	0.627
Recall (dis.)	0.996	0.481
Macro F1	0.976	0.594

The drop is severe: accuracy falls 38.6 percentage points ($98.57\% \rightarrow 60.00\%$), and recall on the diseased class collapses to 48.1%, meaning BoVien misses over half of the genuinely diseased Kotagiri leaves, barely better than chance on this near-balanced (219 healthy / 216 diseased) binary problem. This confirms that the strong in-distribution accuracy reported throughout this paper (Section 6.1) does not, by itself, imply robustness to a truly independent collection site, and is consistent with the OOD-detection results above: whatever ConvNeXt-V2 has learned generalises well to inputs like the ones it was trained on, but far less well to a genuinely new domain within the same task.

Following the fallback strategy motivated by this gap, we test whether a handful of labelled Kotagiri examples can adapt the model without retraining it. Using BoVien’s fine-tuned backbone as a frozen feature extractor, we compute a class prototype (feature centroid) from k randomly drawn support images per class and classify every remaining Kotagiri image by nearest prototype in ProtoNet style, following the same recipe as the within-domain few-shot evaluation in Section 6.4, aver-

aged over 200 random supports per k for stability.

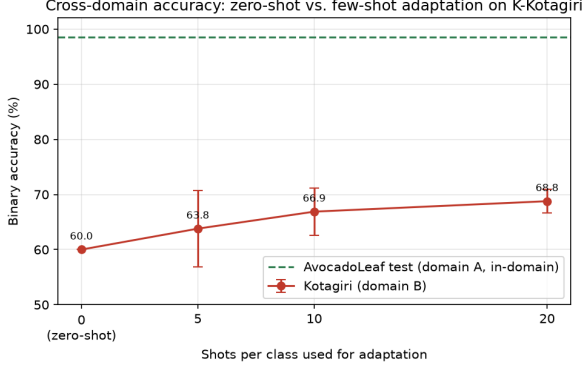


Figure 20: Binary accuracy on K-Kotagiri: zero-shot (BoVien’s own classifier head) versus few-shot nearest-prototype adaptation using BoVien’s fine-tuned backbone as a frozen feature extractor, at 5/10/20 labelled Kotagiri images per class (200 random supports each, error bars are ± 1 std). The dashed line is BoVien’s in-domain (AvocadoLeaf test set) binary accuracy.

Few-shot adaptation helps but does not close the gap: accuracy rises from 60.00% (zero-shot) to 63.80% $\pm$ 6.91% at 5 shots, 66.88% $\pm$ 4.26% at 10 shots, and 68.78% $\pm$ 2.16% at 20 shots per class, a monotonic and increasingly stable improvement, but still 30 points short of the 98.57% in-domain accuracy. Practically, this means that a handful of labelled examples from a new deployment site is worth collecting and does measurably help, but is not, on its own, a substitute for either a larger site-specific labelled set or a properly retrained/fine-tuned model. See Section 7 for how this qualifies the paper’s edge-deployment and generalisation claims.

6.7. Real-time Inference under Realistic Conditions

The architecture comparison in Section 6.1 reports GPU inference latency measured on batched, pre-processed test images, which does not speak to true embedded/mobile hardware, the setting edge deployment claims are ultimately about. To close this gap, we exported BoVien to ONNX and deployed it on four platforms. On Windows, ONNX Runtime runs the model with its CPU execution provider. On macOS (iMac), ONNX Runtime is configured to use its Core ML execution provider when available, falling back to CPU otherwise. On Android, the raw ONNX model is bundled directly into a native app and run with ONNX Runtime for Android’s default CPU execution provider, with no NNAPI or GPU acceleration enabled. On iOS, the ONNX model is instead converted ahead of time to a native Core ML package and run through Apple’s Core ML

runtime. On each platform, one representative image per AvocadoLeaf class (6 images) was displayed full-screen on a second device and presented to that platform’s own camera, with prediction confidence, per-frame latency, and FPS logged live. Figure 21 shows one such capture on the Mac platform, correctly identifying a *moctrang* (powdery mildew) leaf at 59.0% confidence, 14.2 FPS, 36.51 ms latency. These numbers measure the exported classification model alone. The background-removal preprocessing step (Section 3) is not included in this on-device pipeline, so true end-to-end latency including preprocessing would be higher, particularly on mobile hardware.

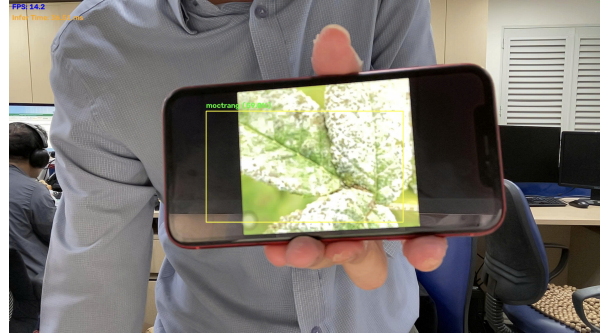


Figure 21: BoVien running live on a Mac (iMac), correctly classifying a phone-displayed *moctrang* (powdery mildew) leaf at 59.0% confidence, 36.51 ms latency (14.2 FPS).

Table 13 reports the mean confidence, FPS, and latency per platform, averaged over the six per-class captures. The full per-class breakdown is in Appendix F. Windows is fastest (33.6 FPS, 15.0 ms), a CPU-only ONNX Runtime session on laptop compute with no mobile-hardware constraint. Among the two genuinely on-device mobile deployments, iOS (native Core ML runtime, with access to Apple’s Neural Engine and GPU) is substantially faster than Android (ONNX Runtime’s CPU execution provider only, no NNAPI or GPU acceleration enabled) at 17.7 FPS versus 2.7 FPS, a more than six-fold gap that reflects the accelerator each deployment path has access to rather than a difference in the underlying model. The Mac falls between the two laptop and two mobile results (14.5 FPS). Mean confidence also varies by platform (61.0% to 84.6%), which is expected. Each platform’s camera, lighting, and capture angle differ, and this reflects real deployment variability rather than a change in the underlying model’s accuracy, which is fixed by the AvocadoLeaf test set result in Table 6. Even the slowest platform (Android, 2.7 FPS) remains well within the range usable for a farmer pointing a phone at a leaf and waiting a fraction of a second for

a diagnosis, though it is far from Windows’s real-time responsiveness.

Table 13: Interactive on-device inference of BoVien (ONNX export) across four platforms, one live capture per AvocadoLeaf class (mean over 6 classes). Windows and macOS run the ONNX model on laptop compute via ONNX Runtime (CPU, and CPU/Core ML respectively). Android runs the same ONNX model natively via ONNX Runtime for Android (CPU only). iOS runs a Core ML conversion of the model natively. *Excludes background-removal preprocessing, see text.*

Platform	Mean conf. (%)	FPS	Mean latency (ms)
Windows	61.0	33.6	15.0
macOS	79.9	14.5	30.6
iOS	84.6	17.7	51.6
Android	76.2	2.7	283.6

7. Discussion

Why ConvNeXt-V2 works well on a fine-grained, non-linearly-separable dataset. Section 4 showed that frozen ImageNet features are not linearly separable for AvocadoLeaf (near-zero Silhouette score, held-out linear-probe accuracy of only 65.48%), reflecting substantial visual overlap between disease categories. The architecture comparison in Section 6.1 is consistent with this picture. Only fine-tuned, non-linear models reach usable accuracy, and among them ConvNeXt-V2 achieves the best trade-off (92.14% accuracy, 0.900 macro F1) with the fewest parameters (3.4M) of any competitive architecture, including its predecessor ConvNeXt-V1 (27.8M). This suggests that, for this task, ConvNeXt-V2’s Global Response Normalisation and design refinements over ConvNeXt-V1 translate into a more sample- and parameter-efficient decision boundary rather than raw capacity being the deciding factor, since several much larger models (ViT, SwinTransformer) do not clearly outperform it.

Few-shot learning narrows but does not close the data-efficiency gap. The same non-linear-separability finding motivated comparing fine-tuning against metric-based few-shot learning (Section 6.4). ProtoNet more than doubles fine-tuning’s accuracy at every shot level tested (e.g. 57.4% vs. 28.6% at 20 shots), confirming that a task-adapted embedding space is far more data-efficient than directly fine-tuning a classifier head on a handful of examples. However, ProtoNet’s absolute accuracy remains well below the >90% achieved with the full training set, so few-shot learning should currently be viewed as a rapid-onboarding aid for a low-label adaptation strategy or new orchards rather than a replacement

for full-scale data collection. RelationNet’s near-chance performance (16–19%, against a 16.7% random baseline) is a notable negative result. Unlike ProtoNet’s fixed distance metric, RelationNet must learn its comparison function from the same limited episodes, and with only 1,302 training-set images available for episodic training this learnable module likely lacks enough data to converge to a useful metric.

The model is reliable at detecting what it doesn’t know, but its raw confidence is only moderately calibrated. The uncertainty analysis (Section 6.5) shows two distinct results that carry different practical implications. Out-of-distribution detection is strong for both far OOD (AUC 0.994) and near OOD (AUC 0.985) inputs, meaning the model can reliably flag non-leaf images and, importantly, leaves from other crop species as “not avocado” rather than silently forcing them into one of the six trained classes, a valuable safety property for a farmer-facing tool. The raw ECE of 0.0625 is above the common 0.05 threshold for good calibration, so raw softmax confidence should not be read as a literal probability of correctness out of the box. However, we show in Section 6.5 that a single fitted temperature parameter ($T = 1.274$) brings test-set ECE down to 0.0193 at no cost to accuracy, so this is a substantially mitigated by temperature scaling, limitation, and confidence scores surfaced to end users should simply be the temperature-scaled ones.

Interpretability evidence is now quantitative and full-test-set, but still leaves failure modes uncharacterised. The LIME explanations in Section 6.2 localise to symptomatic regions (scorched lesions for *chaymepla*, powdery patches for *moctrang*) rather than background or leaf-margin artefacts, and the deletion-metric evaluation across all 280 test images (mean confidence drop 0.649, 95% CI [0.618, 0.678], Appendix D) confirms this is not an artefact of a favourably chosen example. Hiding the ~20% of the image LIME attributes each prediction to collapses the model’s confidence by nearly two-thirds on average. The drop is nonetheless smaller for misclassified images (0.516) than correct ones (0.660) and is lowest for *repsap* (0.416), whose diffuse, non-contiguous damage pattern is a harder fit for a fixed-size superpixel mask. The confusion-matrix error analysis in Section 6.1.3 corroborates this from an independent angle: *repsap* is also the class BoVien most often confuses with *chaymepla* (12.0% of its test images), so its lower LIME faithfulness and its higher misclassification rate point at the same underlying cause, a visually diffuse symptom pattern, rather than two unrelated weaknesses.

A language model is a useful explainer of the

CNN’s decisions, but not a useful automatic error detector. Section 6.3 showed that a small (2B parameter), free, locally-hosted vision-language model can successfully combine the CNN’s prediction with LIME’s visual attribution to produce grounded, farmer-readable explanations. This was the original motivation for adding a language model to the system, and it works, as the generated text correctly references both the attributed leaf region and the class-appropriate visual symptom in the examples inspected (Figure 14). We separately probed a harder use of the same model that we do *not* report success on (Appendix E gives the full protocol and results), namely whether the explainer’s own `supports_cnn_diagnosis` judgement, without ever overriding the CNN, could serve as a free-standing signal for automatically flagging likely CNN mistakes. It reached a precision of 0.40 but a much lower recall (0.21) than a zero-cost baseline that simply flags the top half of the escalated subset by predictive entropy (precision 0.24, recall 0.79), and every mistake it caught was already caught by that entropy baseline. So, at the model scale available under a no-budget constraint, a language model earns its place in this system specifically as an explanation layer on top of an unchanged CNN + LIME pipeline, not as a substitute for the uncertainty estimation the system already has.

Limitations. Several limitations qualify the conclusions above. First, the dataset, while diverse in imaging conditions, was collected in only three provinces of the Central Highlands, and its class sizes range from 165 to 654 images, so both generalisation to other growing regions and performance on the rarest classes (*rep-sap*, *gisar*) should be treated cautiously. Second, the expert quality-control review (Section 3) involved a single expert, so the reported agreement reflects consistency with one reviewer’s judgement rather than genuine inter-expert reliability. A second independent expert would be needed to measure that directly. Third, and separately, the annotator pool itself is thin: three annotators means every image is settled by a 2-1 majority vote at best, and the three-way tie-break procedure defined in Section 3 was never exercised in practice, so it remains an untested part of the protocol rather than a validated one. Fourth, the intrinsic difficulty metrics and linear-probe accuracy were computed on a validation set of only 279 images, which inflates the variance of these estimates. Fifth, beyond the GPU inference latency reported in Section 6.1, Section 6.7 now reports genuine on-device latency on an Android phone (ONNX Runtime, CPU only) and an iPhone (native Core ML), alongside laptop latency on Windows and macOS (both via ONNX Runtime). This substantiates edge-deployment

claims for mobile hardware specifically, though a dedicated embedded board such as a Raspberry Pi remains untested, no GPU/NNAPI acceleration was enabled for the Android measurement, and the reported numbers exclude the background-removal preprocessing step (Section 3), so true end-to-end field latency, especially on Android, would be higher than Table 13 reports. Each platform’s numbers also come from a single live capture per class rather than a sustained session, so they should be read as an indicative first measurement rather than a fully characterised latency distribution. Sixth, the real cross-domain evaluation (Section 6.6.2) is a genuine same-task, independent-site test, but only at the coarse healthy/diseased granularity, since K-Kotagiri does not provide per-disease labels. It does not establish whether BoVien’s specific disease-class predictions (*chaymepla*, *domla*, etc.) generalise across sites, only whether the binary healthy-vs-diseased signal does, and on this coarser task it generalises poorly (60.00% zero-shot accuracy, down from 98.57% in-domain). A second, independently collected 6-class AvocadoLeaf dataset would be needed to test disease-level cross-domain generalisation, and remains future work. Seventh, the annotation guideline assigns a single label per image based on the most severe visible symptom (Section 3), so leaves with co-occurring symptoms (e.g. mealybug damage alongside an unrelated leaf spot) are reduced to one class. This is a source of label ambiguity that the current single-label taxonomy cannot capture, and it likely contributes to the classes with more diffuse or compound symptom patterns being harder to classify and to explain (Section 6.2’s per-class LIME faithfulness results). Eighth, the unfrozen-layer depth used during transfer learning is not held perfectly constant across architecture families (Section 5), a source of protocol variation not disentangled by this study. ResNet-18 is the only architecture affected, and it does not rank near the top of Table 6, so this is unlikely to change the selection of ConvNeXt-V2 but has not been verified by an equal-depth re-run. Ninth, the ablation study (Section 6.1.2) surfaces a discrepancy between the training protocol as described in Section 5, which states that all 15 candidates share the same data-augmentation policy, and the actual checkpoint configuration. The ConvNeXt-V2/BoVien checkpoint reported in Table 6 was in fact trained with augmentation disabled, and spot-checking other candidates’ saved configurations shows this switch was not applied uniformly across the benchmark. We do not have evidence that this changes which architecture ranks best. The ablation shows augmentation *hurts* BoVien specifically, so if anything this makes the benchmark conservative with respect to BoVien rather than favouring it.

We have not re-run the full 15-model benchmark under a corrected, uniform augmentation switch before this submission, since doing so does not change which architecture the paper selects and the compute cost of 15 additional single-seed training runs was better spent, within this study’s timeline, on the real cross-domain evaluation of Section 6.6.2. A fully protocol-matched re-run remains the direct way to close this gap and is prioritised as future work. Tenth, every architecture in Table 6, including the four ablation configurations, was trained once, with a single random seed. The bootstrap intervals in Appendix C and Appendix C.1 quantify uncertainty from the finite 280-image test set, not from training-run-to-training-run variance. A different seed could shift any single model’s accuracy by an amount this study does not measure. This matters most for the top six models in Table 6, which are already statistically tied on test-set noise alone, so seed variance is a second, unmeasured source of uncertainty on top of that tie, not a separate concern that only affects lower-ranked models. Multi-seed re-training of at least the top six candidates is left as future work to quantify this directly.

8. Conclusion and Future Work

This paper introduced AvocadoLeaf, a 1,861-image dataset of five field-collected avocado leaf disease categories and healthy leaves, and used it to benchmark 15 lightweight and efficient architectures for edge-deployable disease classification. ConvNeXt-V2 offered the best accuracy-efficiency trade-off, reaching 92.14% accuracy and 0.900 macro F1 with only 3.4M parameters and 2.30 ms per-image inference on a single GPU, though its accuracy is statistically tied with five other architectures (Section 6.1, Appendix C). Metric-based few-shot learning with a frozen ConvNeXt-V2 backbone more than doubled the accuracy of conventional fine-tuning at every shot level tested (e.g. 57.4% vs. 28.6% at 20 shots), and uncertainty estimation showed that the model reliably separates in-distribution avocado leaves from both unrelated images (far OOD AUC 0.994) and leaves of other crop species (near OOD AUC 0.985), while a full-test-set LIME deletion-metric evaluation confirmed that its predictions localise to disease-relevant leaf regions rather than background artefacts (mean confidence drop 0.649 when the attributed region is hidden, 95% CI [0.618, 0.678], $n = 280$). Building on this, we combined the CNN’s prediction with LIME’s visual attribution and a small, free, locally-hosted vision-language model to generate grounded, natural-language explanations of individual

predictions, while keeping the CNN as the sole decision-maker (Section 6.3). Together, these results indicate that an accurate, compact, and interpretable classifier can be built for avocado disease diagnosis under realistic, field-collected conditions.

We additionally examined several deployment-facing questions raised by these results. First, a naive-deployment failure-mode analysis (Section 6.6) showed that ConvNeXt-V2’s advantage over the lighter candidates is not limited to in-distribution accuracy. Without an uncertainty gate, MobileNet-V2 confidently (>0.9) misclassifies 22–30% of domain-shifted, non-avocado images as a specific disease, versus 0–1.5% for ConvNeXt-V2, making the entropy-based uncertainty gate (Section 6.5) a necessary companion to any lighter model chosen for deployment rather than an optional add-on. Second, and more directly, we obtained a genuine independent, same-task cross-domain evaluation (Section 6.6.2) using K-Kotagiri (2024), a separately collected avocado leaf dataset from India: trained only on AvocadoLeaf and evaluated zero-shot at the healthy-vs-diseased level (the only granularity Kotagiri’s labels support), accuracy falls from 98.57% in-domain to 60.00%, and a few-shot adaptation fallback using as few as 20 labelled target-domain images per class recovers only part of this gap (up to 68.78%). This is the single most important caveat on this paper’s generalisation claims: strong in-distribution accuracy does not, on its own, indicate that BoVien is ready to deploy at a new site without either site-specific labelled data or further adaptation. Third, we deployed BoVien live across four platforms (Section 6.7), running natively on-device on Android (ONNX Runtime, CPU only) and iOS (native Core ML) as well as via ONNX Runtime on Windows and macOS laptop compute, obtaining real per-platform confidence, FPS, and latency numbers from a live camera rather than only a batched GPU benchmark. Fourth, beyond identifying gaps, we resolved one of them directly: post-hoc temperature scaling reduces test-set ECE from 0.0311 to 0.0193 (Section 6.5) at no cost to accuracy. We also characterised, rather than merely flagged, where BoVien’s remaining errors concentrate: a confusion-matrix-driven error analysis (Section 6.1.3) shows they cluster in the three lowest-support classes and in three visually-motivated confusion pairs (*gisat*↔*moctrang*, *repsapl*/*moctrang*→*chaymepla*), consistent with the LIME faithfulness results (Section 6.2).

At the same time, Section 7 identified concrete gaps that remain open. A dedicated embedded board such as a Raspberry Pi remains untested, and the on-device numbers above exclude the background-removal preprocessing step, so true end-to-end field latency would be some-

what higher, particularly on Android. The real cross-domain evaluation above is same-task but binary-only, so disease-level (6-class) cross-domain generalisation is still untested. The ablation study (Section 6.1.2) also surfaced a documentation/protocol discrepancy in the shared augmentation policy across the 15-architecture benchmark. We also tested, and found unsuccessful, a harder use of the same language model beyond explanation generation, using its explanation-derived judgement as a standalone error-detection signal, which underperformed the CNN’s own existing uncertainty estimation (Section 7, Appendix E). Future work will therefore prioritise (1) Raspberry-Pi/embedded-board latency measurement that includes the background-removal step, (2) collecting an independent, same-task, *6-class* AvocadoLeaf dataset to extend the binary cross-domain result above to disease-level generalisation, and (3) a fully protocol-matched re-run of the 15-architecture benchmark under a uniform augmentation policy to close the discrepancy noted above. Beyond these, we plan to expand the dataset to more growing regions, investigate why RelationNet failed to learn a useful metric in the few-shot setting in contrast to ProtoNet, validate the VLM-generated explanations against expert (agronomist) judgement rather than only the automated correctness proxy used here, and conduct a more extensive, systematic exploratory data analysis to guide future data collection toward the classes and conditions the model currently finds hardest.

CRedit authorship contribution statement

Long Nguyen-Hoang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Visualization. **Dung Ho-Tan:** Data curation, Investigation, Validation, Writing – review & editing. **Tien Minh-Do:** Data curation, Investigation, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The AvocadoLeaf dataset and the code used to produce the results in this paper will be made publicly available upon publication.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Raw Data Quality Control and Agreement Calculation

Appendix A.1. Per-Annotator Performance vs. Expert

Table A.14 reports the number of images on which each annotator agreed with the independent expert.

Table A.14: Individual annotator agreement with the expert on the 279-image review set.

Annotator	Correct	Incorrect	Accuracy
Annotator 1	236	43	0.846
Annotator 2	235	44	0.842
Annotator 3	236	43	0.846

On average, an individual annotator agreed with the expert on roughly 84.5% of the images.

Appendix A.2. Consensus (Majority Vote) vs. Expert

The final label for each image was obtained by majority voting among the three annotators. Table A.15 shows the confusion matrix between this consensus label and the expert label.

Table A.15: Confusion matrix, consensus (majority vote) vs. expert on the 279-image review set.

	binhthuong	chaymepla	domla	gisat	moctrang	repsap	Total
binhthuong	43	1	1	1	0	0	46
chaymepla	1	43	1	0	1	0	46
domla	1	1	43	0	0	1	46
gisat	1	0	0	44	1	1	47
moctrang	0	1	0	1	44	1	47
repsap	0	0	1	1	1	44	47
Total	46	46	46	47	47	47	279

Appendix A.3. Calculation of Agreement Metrics

From the confusion matrix:

- Observed agreement** (p_o) = sum of diagonal elements / total images = $(43 + 43 + 43 + 44 + 44 + 44)/279 = 261/279 \approx 0.9355$, reported as 0.94.
- Expected agreement by chance** (p_e) is computed from the marginal distributions. Row totals R and column totals C are identical, both equal to $(46, 46, 46, 47, 47, 47)$. Hence

$$p_e = \frac{\sum_{i=1}^6 R_i C_i}{N^2} = \frac{3 \times 46^2 + 3 \times 47^2}{279^2} = \frac{3 \times 2116 + 3 \times 2209}{77841} = \frac{12975}{77841} \approx 0.1667. \quad (\text{A.1})$$

- Cohen's Kappa** is then

$$\kappa = \frac{p_o - p_e}{1 - p_e} = \frac{0.9355 - 0.1667}{1 - 0.1667} = \frac{0.7688}{0.8333} \approx 0.9226, \quad (\text{A.2})$$

which is rounded to 0.92 in the main text.

These results confirm that the three-annotator consensus achieves very high agreement with the expert (93.5% observed agreement, $\kappa = 0.92$), validating the annotation protocol.

Appendix B. Extended Difficulty Analysis

This appendix provides supplementary details for the difficulty analysis presented in Section 4, Section 4.3.

Appendix B.1. Software and Implementation Details

All experiments were implemented in Python 3.10 using PyTorch 2.0.0 for feature extraction. The t-SNE projection was performed using the scikit-learn implementation with perplexity 30, 1000 iterations, and a random state of 42. Logistic regression was implemented via scikit-learn’s LogisticRegression with the LBFGS solver and a maximum of 1000 iterations to ensure convergence.

Appendix B.2. Mathematical Definition of Metrics

For completeness, we restate the three clustering evaluation metrics used in the main paper.

Silhouette Score. For a sample i assigned to cluster C_i , let $a(i)$ be the mean distance between i and all other points in the same cluster, and $b(i)$ the mean distance between i and all points in the nearest neighbouring cluster. The Silhouette score of i is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

The overall Silhouette score is the average of $s(i)$ over all samples. The score ranges from -1 (poor clustering) to $+1$ (excellent clustering). A value near zero indicates substantial overlap between clusters.

Davies–Bouldin Index (DBI). For each cluster C_i , define its scatter S_i as the average distance of points in C_i to its centroid. For two clusters C_i and C_j , let d_{ij} be the distance between their centroids. The DBI is:

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{d_{ij}} \right),$$

where k is the number of clusters. Lower DBI values imply better separation. The index has no fixed upper bound and is best interpreted relative to other clustering configurations on the same data rather than against a universal cut-off.

Intra/Inter-class Distance Ratio. Let $\mathbf{c}_g$ denote the centroid of class g . The average intra-class distance is:

$$D_{\text{intra}} = \frac{1}{k} \sum_{g=1}^k \frac{1}{|C_g|} \sum_{\mathbf{x} \in C_g} \|\mathbf{x} - \mathbf{c}_g\|_2.$$

The average inter-class distance is defined as the mean pairwise distance between centroids:

$$D_{\text{inter}} = \frac{2}{k(k-1)} \sum_{1 \leq i < j \leq k} \|\mathbf{c}_i - \mathbf{c}_j\|_2.$$

The ratio $R = D_{\text{intra}}/D_{\text{inter}}$ quantifies the trade-off between within-class compactness and between-class separation. A ratio $R \geq 1$ suggests that classes are not well separated.

Appendix C. Bootstrap Confidence Intervals and Significance Testing

This appendix provides the full statistical analysis supporting the architecture comparison in Section 6. Because each architecture was trained once (single seed) on the same 1,302-image training set, we do not have multiple independent training runs to estimate between-run variance. We instead quantify the uncertainty arising from the finite 280-image test set via non-parametric bootstrap resampling of the already-computed test predictions (no retraining involved). For each model, the test set is resampled with replacement 1,000 times, accuracy and macro F1 are recomputed on each resample, and the 2.5th/97.5th percentiles of the resulting distributions are reported as a 95% confidence interval. To compare the leaderboard leader (ConvNeXt-V2) against every other architecture, we use *paired* bootstrap resampling, where the same resampled indices are applied to both models’ predictions on each draw, which controls for the fact that some test images are inherently easier or harder than others, and report the 95% CI of the accuracy difference together with a two-sided bootstrap p -value (the fraction of resamples in which the sign of the difference flips).

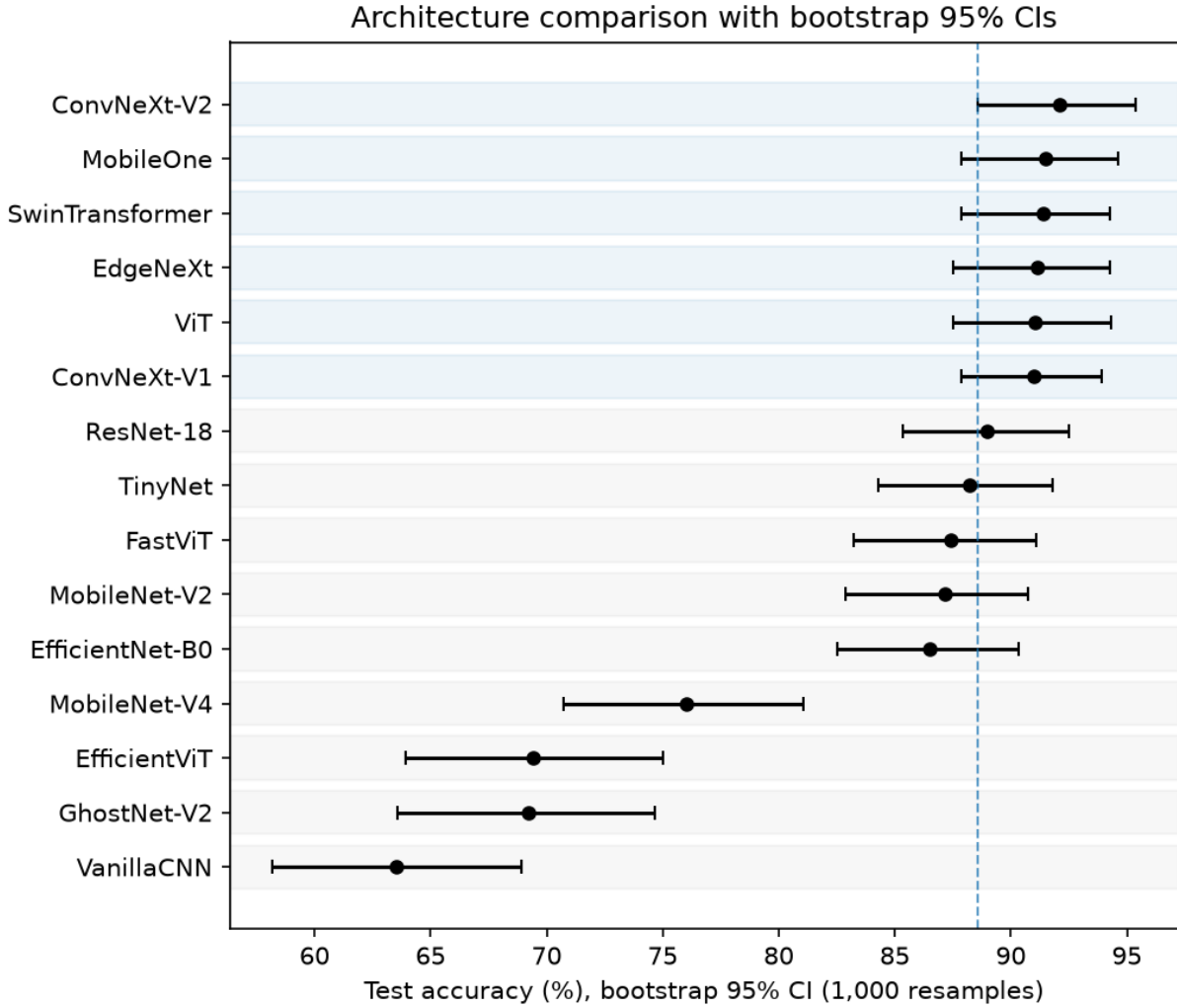


Figure C.22: Bootstrap 95% CI forest plot of test accuracy, sorted by point estimate. Blue rows are the six models statistically tied with ConvNeXt-V2; grey rows are significantly below it ($p < 0.05$). The dashed line marks ConvNeXt-V2's lower CI bound.

Figure C.22 visualises the same accuracy confidence intervals as a forest plot, sorted by point estimate. The dashed line marks ConvNeXt-V2's lower CI bound. The six models shaded blue (ConvNeXt-V2 and the five statistically tied challengers) all have error bars crossing this line, while every model below them does not, showing the "tied top-6, then a real gap" structure at a glance.

Interpretation. ConvNeXt-V2's accuracy advantage over the next five models on the leaderboard (SwinTransformer, MobileOne, ViT, EdgeNeXt, ConvNeXt-V1) is *not* statistically significant at the 280-image test set size ($p > 0.5$ in all five cases; 95% CIs on the difference all include zero). ConvNeXt-V2 is significantly more accurate than every model ranked seventh or lower ($p < 0.05$). This means the ranking of the top six models by point-estimate accuracy alone should not be over-interpreted as a strict ordering; the top six are statistically indistinguishable in accuracy on this test set. ConvNeXt-V2 remains the preferred choice not because it is proven to be the single most accurate architecture, but because among this statistically-tied top group, it has by far the smallest parameter count (3.4M against 4.3 to 85.8M for the other five) and the lowest inference latency, which is what Section 6's efficiency analysis and edge-deployment motivation are based on.

Table C.16: Bootstrap 95% confidence intervals (1,000 resamples) for test accuracy and macro F1 of all 15 architectures. The reported point estimates are the *mean* of the 1,000 bootstrap resamples, not the single test-set value in Table 6; the two differ by at most 0.11 pp / 0.004 F1 for every model, well within the reported intervals, and the CI, not the point estimate, is what supports the statistical claims in this appendix.

Model	Accuracy (%) [95% CI]	Macro F1 [95% CI]
ConvNeXt-V2	92.11 [88.57, 95.36]	0.898 [0.853, 0.940]
MobileOne	91.49 [87.86, 94.64]	0.892 [0.847, 0.931]
SwinTransformer	91.39 [87.86, 94.29]	0.898 [0.855, 0.934]
EdgeNeXt	91.13 [87.50, 94.29]	0.876 [0.827, 0.919]
ViT	91.06 [87.50, 94.29]	0.903 [0.864, 0.940]
ConvNeXt-V1	90.98 [87.85, 93.94]	0.893 [0.850, 0.930]
ResNet-18	88.98 [85.36, 92.50]	0.862 [0.817, 0.908]
TinyNet	88.19 [84.29, 91.79]	0.862 [0.815, 0.904]
FastViT	87.40 [83.21, 91.07]	0.850 [0.801, 0.895]
MobileNet-V2	87.15 [82.86, 90.71]	0.845 [0.794, 0.891]
EfficientNet-B0	86.52 [82.50, 90.36]	0.836 [0.780, 0.884]
MobileNet-V4	76.03 [70.71, 81.07]	0.720 [0.656, 0.777]
EfficientViT	69.40 [63.93, 75.00]	0.648 [0.583, 0.714]
GhostNet-V2	69.20 [63.57, 74.64]	0.567 [0.497, 0.640]
VanillaCNN	63.51 [58.21, 68.94]	0.619 [0.554, 0.681]

Table C.17: Paired bootstrap comparison of ConvNeXt-V2 against every other architecture (accuracy difference, 1,000 resamples).

Comparison (ConvNeXt-V2 – X)	Diff (pp)	95% CI (pp)	<i>p</i>
SwinTransformer	+0.68	[−2.50, +3.93]	0.768
MobileOne	+0.72	[−2.50, +3.93]	0.722
ViT	+1.06	[−2.14, +4.29]	0.570
EdgeNeXt	+1.13	[−2.14, +4.29]	0.564
ConvNeXt-V1	+1.13	[−1.79, +3.93]	0.526
ResNet-18	+3.20	[+0.36, +6.07]	0.030
TinyNet	+4.00	[+0.36, +7.50]	0.034
FastViT	+4.60	[+1.07, +8.57]	0.014
MobileNet-V2	+5.09	[+1.79, +8.21]	0.002
EfficientNet-B0	+5.68	[+1.79, +10.00]	0.002
MobileNet-V4	+16.14	[+11.42, +21.07]	<0.001
GhostNet-V2	+22.73	[+17.14, +28.21]	<0.001
EfficientViT	+22.91	[+17.50, +28.21]	<0.001
VanillaCNN	+28.64	[+23.21, +34.29]	<0.001

Appendix C.1. Ablation Study Significance Testing

The same methodology is applied to the four-configuration ablation of Section 6.1.2, using the per-sample predictions of each of the four BoVien checkpoints on the same 280-image test set.

Table C.18: Bootstrap 95% confidence intervals (1,000 resamples) for the ablation study’s four configurations.

Configuration	Accuracy (%) [95% CI]	Macro F1 [95% CI]
No augmentation, no class-weight	92.11 [88.93, 95.00]	0.898 [0.855, 0.935]
Augmentation, class-weighted	91.47 [88.21, 94.29]	0.894 [0.849, 0.933]
No augmentation, class-weighted	90.70 [86.79, 94.29]	0.886 [0.837, 0.928]
Augmentation, no class-weight	88.99 [85.00, 92.50]	0.863 [0.815, 0.907]

Interpretation. None of the three pairwise differences reach the conventional $p < 0.05$ threshold at this test set size, including the largest gap (no augmentation, no class-weight, versus augmentation, no class-weight, +3.31pp,

Table C.19: Paired bootstrap comparison of the best ablation configuration (no augmentation, no class-weight) against the other three (accuracy difference, 1,000 resamples).

Comparison (best – X)	Diff (pp)	95% CI (pp)	p
Augmentation, class-weighted	+0.70	[−1.79, +3.21]	0.692
No augmentation, class-weighted	+1.37	[−1.07, +3.93]	0.336
Augmentation, no class-weight	+3.31	[+0.00, +6.44]	0.054

$p = 0.054$), whose 95% CI lower bound sits at essentially zero. The ablation study’s ranking should therefore be read as suggestive rather than conclusive. The direction is consistent (the no-augmentation configuration is the point-estimate best in three independent comparisons) and matches the sign of the discrepancy discussed in Section 7, but a 280-image test set and single-seed training runs cannot establish statistical significance for effects of this size (roughly 1 to 3 accuracy points). A future protocol-matched re-run with multiple random seeds per configuration, as noted in Section 7, would be needed to confirm the ranking rather than only its direction.

Appendix D. LIME Faithfulness Evaluation Methodology and Per-Class Results

This appendix details the deletion-metric protocol summarised in Section 6 (Model Interpretability) and reports the per-class breakdown.

Appendix D.1. Protocol

For every image in the 280-image test set, four steps are applied. (1) The model’s baseline confidence in its own predicted class is recorded; (2) LIME is run with the same settings used for the qualitative examples (Quickshift segmentation, 500 perturbed samples, top 10 superpixels); (3) the superpixels LIME attributes to the predicted class are replaced with the masking colour used during LIME’s own sampling (black, consistent with `hide_color=0` in the LIME configuration), and the model is re-run on this perturbed image; (4) the confidence drop is the baseline confidence minus the perturbed-image confidence, both measured for the *same* (originally predicted) class. This is the standard “deletion” faithfulness metric used in the XAI literature, in the spirit of the masking-based evaluation introduced for saliency methods by RISE (Petsiuk et al., 2018). An explanation that faithfully identifies the pixels the model relies on should produce a large confidence drop when those pixels are removed, whereas an unfaithful or random explanation should produce a drop close to what removing a random region of the same size would cause. A 95% confidence interval for the mean confidence drop is computed via bootstrap resampling of the 280 per-image drops (1,000 resamples, percentile method), identical in spirit to the procedure in Appendix C.

Appendix D.2. Per-Class Results

Table D.20 breaks the mean confidence drop down by true class. The two classes with the most spatially localised, high-contrast symptoms in our annotation guideline, *domla* (leaf spot) and *binhthuong* (healthy, where the model instead grounds its decision in the absence of any lesion pattern), show the largest drops. *repsap* (mealybug damage) shows the smallest drop, consistent with the qualitative observation in Section 6 that its damage pattern is diffuse and stippled rather than a single contiguous lesion, making it harder for a fixed-size top-10-superpixel mask to capture all of the evidence the model uses.

Appendix E. LLM/VLM Integration Protocol and Results

This appendix details the VLM-explainer protocol summarised in Section 7, including a secondary analysis of whether its output can double as an automatic error-detection signal. The experiment uses ConvNeXt-V2 as the base classifier, runs on the 140 least-confident images of the 280-image test set (top 50% by MC-Dropout predictive entropy, as in Section 6’s uncertainty analysis), and uses only a small, locally-hosted, zero-API-cost model (Qwen2-VL-2B-Instruct), reflecting a no-budget resource constraint rather than an upper bound on what larger, paid models might achieve.

Table D.20: Mean LIME deletion-metric confidence drop by true class (280 test images total).

Class	Mean drop	Std	n
domla	0.769	0.183	98
binhthuong	0.695	0.216	53
moctrang	0.657	0.207	30
gisat	0.589	0.301	27
chaymepla	0.498	0.247	47
repsap	0.416	0.249	25

Appendix E.1. Explanation Protocol: VLM as Explainer

The vision-language model is given the CNN’s already-decided prediction, the raw image, and the LIME overlay image (boundary-marked highlighted region), and is asked only to output a boolean `supports_cnn_diagnosis` judgement plus free-text visual evidence, never a competing class label; the CNN’s decision is never overridden. `final_pred` is hard-set to the CNN’s prediction for every image in this design (verified to hold for 100% of rows). Of the 140 escalated images, 125 yielded a valid, parseable `supports_cnn_diagnosis` boolean; the remaining 15 are excluded from the scoring below since neither flag can be scored without a VLM output to compare against. We then treat `supports_cnn_diagnosis = False` as a zero-cost-to-CNN “the CNN might be wrong” flag and score it against the CNN’s actual (test-set-label-derived) correctness, comparing it to a free baseline that flags the top half of this same 125-image scored subset by predictive entropy, so that both flags are evaluated on an identical denominator.

Table E.21: VLM disagreement flag vs. a free entropy-threshold baseline, scored against actual CNN mistakes on the 125-image subset of the 140 escalated images that yielded a parseable VLM output (19 were actual CNN mistakes).

Flag	Precision	Recall	F1
VLM (<code>supports_cnn_diagnosis = False</code>)	0.40	0.21	0.28
Entropy threshold (top half, free)	0.24	0.79	0.37

Appendix F. Real-time On-device Deployment, Per-class Results

This appendix reports the full per-class breakdown behind the platform-level means in Table 13 (Section 6.7). For each of the four platforms, one representative image per AvocadoLeaf class was displayed full-screen on a second device and presented to that platform’s own camera, with the exported BoVien model, run via ONNX Runtime (Windows, macOS, Android) or a native Core ML conversion (iOS), producing one live prediction logged per class.

The per-class confidence figures should not be read as a second accuracy measurement. Each row is a single live capture per class per platform, so unlike Table 6’s 280-image test-set evaluation, a low confidence value here (e.g. Android’s 45.0% on *repsap*) reflects one particular camera angle, lighting condition, and phone-screen glare on a single frame, not a per-class accuracy estimate. This appendix logs confidence and latency only, not whether the predicted class matched the displayed one on each of the 24 captures.

The VLM flag reaches higher precision but much lower recall than the free entropy baseline, and critically, all 4 CNN mistakes the VLM flag caught were also caught by the entropy baseline (0 complementary detections out of 19 actual mistakes); that is, the VLM’s grounded, image-based explanation added no error-detection information beyond what the CNN’s own predictive entropy already provides at no extra inference cost. Qualitative inspection of the generated explanations also found instances of unfaithful reasoning, e.g. one case where the VLM justified a *correct binhthuong* (healthy) prediction by describing rust-coloured spots, the symptom description belonging to a different class (*gisat*), rather than reasoning from what was actually visible, underscoring that the free-text explanations should not be taken as reliably faithful without further validation.

Table F.22: Per-class confidence, FPS, and latency for the real-time on-device deployment of BoVien, one live capture per class per platform.

Platform	Class	Conf. (%)	FPS	Latency (ms)
Windows	binhthuong	85.6	35.2	15.51
Windows	chaymepla	74.7	37.6	13.59
Windows	domla	35.7	28.9	19.00
Windows	gisat	68.0	33.3	14.00
Windows	moctrang	65.3	36.3	13.00
Windows	repsap	36.7	30.3	15.01
macOS	binhthuong	95.8	14.3	31.60
macOS	chaymepla	81.0	14.9	31.41
macOS	domla	71.9	15.1	30.19
macOS	gisat	85.2	14.6	28.21
macOS	moctrang	70.2	13.4	30.90
macOS	repsap	75.5	14.6	31.40
iOS	binhthuong	98.8	18.2	42.10
iOS	chaymepla	90.2	17.3	55.30
iOS	domla	92.1	18.2	52.60
iOS	gisat	86.1	18.5	52.10
iOS	moctrang	81.0	16.2	53.90
iOS	repsap	59.5	17.6	53.40
Android	binhthuong	89.6	2.7	243.09
Android	chaymepla	80.7	2.6	274.34
Android	domla	93.0	2.5	350.30
Android	gisat	92.9	2.7	272.27
Android	moctrang	55.7	2.9	270.02
Android	repsap	45.0	2.6	291.69

References

- Argüeso, D., Picón, A., Irusta, U., Medela, A., San-Emeterio, M.G., Bereciartua, A., Alvarez-Gila, A., 2020. Few-shot learning approach for plant disease classification using images taken in the field. *Computers and Electronics in Agriculture* 175, 105542.
- Atila, Ü., Uçar, M., Akyol, K., Uçar, E., 2021. Plant leaf disease classification using efficientnet deep learning model. *Ecological Informatics* 61, 101182.
- Awais, M., Alharthi, A.H.S.A., Kumar, A., Cholakkal, H., Anwer, R.M., 2025. Agrogpt: Efficient agricultural vision-language model with expert tuning, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 5687–5696.
- Barbedo, J.G.A., 2019. Plant disease identification from individual lesions and spots using deep learning. *Biosystems Engineering* 180, 96–107.
- Davies, D.L., Bouldin, D.W., 1979. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence PAMI-1*, 224–227.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N., 2021. An image is worth 16x16 words: Transformers for image recognition at scale, in: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Finn, C., Abbeel, P., Levine, S., 2017. Model-agnostic meta-learning for fast adaptation of deep networks, in: *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 1126–1135.

- Gal, Y., Ghahramani, Z., 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 1050–1059.
- Gonzalez, J., Gonzalez, A.G.C., Gonzalez, R.A.M., 2020. Identifying out-of-distribution samples in plant disease classification using deep ensembles, in: Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8.
- Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321–1330.
- Han, K., Wang, Y., Zhang, Q., Zhang, W., Xu, C., Zhang, T., 2020. Model rubik’s cube: Twisting resolution, depth and width for tinynets, in: Advances in Neural Information Processing Systems (NeurIPS).
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778.
- Hossain, M.Z., Samu, M.S.S., Islam, M.R., Ansary, M.S., 2026. A two-stage multitask vision-language framework for explainable crop disease visual question answering. arXiv preprint arXiv:2601.05143 .
- Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H., 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 .
- Hughes, D.P., Salathé, M., 2015. An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv preprint arXiv:1511.08060 .
- K-Kotagiri (NRB), 2024. [dataset] avocado leaf dataset. <https://data.mendeley.com/datasets/6zy6wxhf2v/1>. doi:10.17632/6zy6wxhf2v.1.
- Lakshminarayanan, B., Pritzel, A., Blundell, C., 2017. Simple and scalable predictive uncertainty estimation using deep ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 6405–6416.
- Landis, J.R., Koch, G.G., 1977. The measurement of observer agreement for categorical data. Biometrics 33, 159–174.
- Liu, X., Peng, H., Zheng, N., Yang, Y., Hu, H., Yuan, Y., 2023. EfficientViT: Memory efficient vision transformer with cascaded group attention, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14420–14430.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B., 2021. Swin transformer: Hierarchical vision transformer using shifted windows, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 10012–10022.
- Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S., 2022. A convnet for the 2020s, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 11976–11986.
- van der Maaten, L., Hinton, G., 2008. Visualizing data using t-sne. Journal of Machine Learning Research 9, 2579–2605.
- Maaz, M., Shaker, A., Cholakkal, H., Khan, S., Zamir, S.W., Anwer, R.M., Khan, F.S., 2022. EdgeNeXt: Efficiently amalgamated cnn-transformer architecture for mobile vision applications, in: Proceedings of the European Conference on Computer Vision Workshops (ECCVW), pp. 3–20.
- Mahmood, H., Yu, Y., Anwer, R., 2026. Agrichain: Visually-grounded expert-verified reasoning for interpretable agricultural vision-language models, in: Proceedings of the Fifteenth Language Resources and Evaluation Conference (LREC), ELRA Language Resource Association. pp. 2268–2276. doi:10.63317/2bv5k9hnduop.
- Mohanty, S.P., Hughes, D.P., Salathé, M., 2016. Using deep learning for image-based plant disease detection. Frontiers in Plant Science 7, 1419.

- Mustofa, S., Munna, M.M.H., Emon, Y.R., Rabbany, G., Ahad, M.T., 2023. A comprehensive review on plant leaf disease detection using deep learning. *arXiv preprint arXiv:2308.14087*.
- Petsiuk, V., Das, A., Saenko, K., 2018. RISE: Randomized input sampling for explanation of black-box models, in: *Proceedings of the British Machine Vision Conference (BMVC)*, p. 151.
- Qin, D., Leichner, C., Delakis, M., Fornoni, M., Luo, S., Yang, F., Wang, W., Banbury, C., Ye, C., Akin, B., Aggarwal, V., Zhu, T., Moro, D., Howard, A., 2024. Mobilenetv4: Universal models for the mobile ecosystem, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 78–96.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. “why should I trust you?”: Explaining the predictions of any classifier, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1135–1144.
- Rousseeuw, P.J., 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* 20, 53–65.
- Sakib, S.N., Haque, N., Hossain, M.Z., Arman, S.E., 2025. Plantexpertvqa: A visual question answering dataset for benchmarking vision-language models in plant science. *arXiv preprint arXiv:2508.17117*.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C., 2018. Mobilenetv2: Inverted residuals and linear bottlenecks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520.
- Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization, in: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626.
- Sharma, P., Berwal, Y.P.S., Ghai, W., 2020. Performance analysis of deep learning cnn models for disease detection in plants using image segmentation. *Information Processing in Agriculture* 7, 566–574.
- Shoaib, M., Shah, B., El-Sappagh, S., Ali, A., Ullah, A., Alenezi, F., Gechev, T., Hussain, T., Ali, F., 2023. An advanced deep learning models-based plant disease detection: A review of recent research. *Frontiers in Plant Science* 14, 1158933.
- Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., Batra, N., 2020. PlantDoc: A dataset for visual plant disease detection, in: *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, pp. 249–253.
- Snell, J., Swersky, K., Zemel, R.S., 2017. Prototypical networks for few-shot learning, in: *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4077–4087.
- Sun, J., 2024. Cross-dataset evaluation of deep learning models for plant pest and disease diagnosis. *bioRxiv preprint doi:10.1101/2024.10.07.617111*.
- Sung, F., Yang, Y., Zhang, L., Xiang, T., Torr, P.H.S., Hospedales, T.M., 2018. Learning to compare: Relation network for few-shot learning, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1199–1208.
- Tan, M., Le, Q.V., 2019. Efficientnet: Rethinking model scaling for convolutional neural networks, in: *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6105–6114.
- Tang, Y., Han, K., Guo, J., Xu, C., Xu, C., Wang, Y., 2022. GhostNetV2: Enhance cheap operation with long-range attention, in: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Vasu, P.K.A., Gabriel, J., Zhu, J., Tuzel, O., Ranjan, A., 2023a. FastViT: A fast hybrid vision transformer using structural reparameterization, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 5785–5795.

- Vasu, P.K.A., Gabriel, J., Zhu, J., Tuzel, O., Ranjan, A., 2023b. MobileOne: An improved one millisecond mobile backbone, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7907–7917.
- Wang, C., Zhou, J., Zhang, Y., Wu, H., Zhao, C., 2022. A plant disease recognition method based on fusion of images and graph structure text. *Frontiers in Plant Science* 13, 850258.
- Wang, C., Zhou, J., Zhao, C., Li, J., Teng, G., Wu, H., 2021. Few-shot vegetable disease recognition model based on image text collaborative representation learning. *Computers and Electronics in Agriculture* 184, 106098.
- Woo, S., Debnath, S., Hu, R., Chen, X., Liu, Z., Kweon, I.S., Xie, S., 2023. ConvNeXt V2: Co-designing and scaling convnets with masked autoencoders, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16133–16142.
- Wu, X., Fan, X., Luo, P., Choudhury, S.D., Tjahjadi, T., Hu, C., 2023. From laboratory to field: Unsupervised domain adaptation for plant disease recognition in the wild. *Plant Phenomics* 5, 0038.
- Yang, S., Feng, Q., Zhang, J., Yang, W., Zhou, W., Yan, W., 2024. From laboratory to field: cross-domain few-shot learning for crop disease identification in the field. *Frontiers in Plant Science* 15, 1434222.